

实验 1：机器学习实验环境搭建与约会网站配对效果判定

一：实验目的

- 1、掌握 Python+Anaconda+PyCharm 的安装方法；
- 2、掌握 Anaconda+PyCharm 开发环境的搭建；
- 3、熟练使用 Anaconda+PyCharm 的编程环境；
- 4、了解 k-近邻算法的程序流程；
- 5、掌握 k-近邻算法在约会网站配对项目中预测流程及含义；
- 6、掌握算法的应用。

二：实验环境

- 1、系统：windows 7、windows 10 或 windows 11
- 2、软件：Python、Anaconda、PyCharm

三：实验内容

(一)Python+Anaconda+PyCharm 的安装和基本使用(1-23 页)

1、环境安装过程

(1) 在 Windows 环境下的 Anaconda 下载

一种可以到 Anaconda 官方网站 <https://www.anaconda.com/>，按照操作系统类型获取相应的安装包；

Anaconda3-2020.11-MacOSX-x86_64.pkg	435.3M	2020-11-18 16:45:35	2f96bb47eb5a949da6f99a71d7d66420
Anaconda3-2020.11-MacOSX-x86_64.sh	427.8M	2020-11-18 16:45:35	918de9a9936908fe62514e0ca6873a21
Anaconda3-2020.11-Windows-x86.exe	403.0M	2020-11-18 16:45:34	ca1f6f3e75eb346f5ab2d87bab005878
Anaconda3-2020.11-Windows-x86_64.exe	457.2M	2020-11-18 16:45:34	0841ffcb927a3c2edbd682520f52e546
Anaconda3-2021.04-Linux-aarch64.sh	407.6M	2021-05-07 10:44:23	14f48f5d1310478b11940a3b96eec7b6
Anaconda3-2021.04-Linux-ppc64le.sh	285.3M	2021-05-10 14:23:03	e5c8220526b95293e669734f91194acc
Anaconda3-2021.04-Linux-s390x.sh	291.7M	2021-05-07 10:45:26	e61fac26bf61bc5c3e3c1a93abc4d8e2
Anaconda3-2021.04-Linux-x86_64.sh	539.9M	2021-05-10 14:21:09	230f2c3c343ee58073bf41bd896dd76c
Anaconda3-2021.04-MacOSX-x86_64.pkg	436.9M	2021-05-10 14:21:10	e2aabaad1dc1a4d4a4fc01281fc2a34c7
Anaconda3-2021.04-MacOSX-x86_64.sh	429.3M	2021-05-10 14:21:08	3caed29ad5564b3567676504669342dc
Anaconda3-2021.04-Windows-x86.exe	405.0M	2021-05-10 14:21:09	7adeb27de653a970476c374408342954

另一种可以到开源软件镜像站下载，如清华大学开源软件镜像站、阿里开源镜像站、网易开源镜像站等。

注意：若内存有限，可下载 miniconda 版本，长期学习者不建议

Windows 安装程序

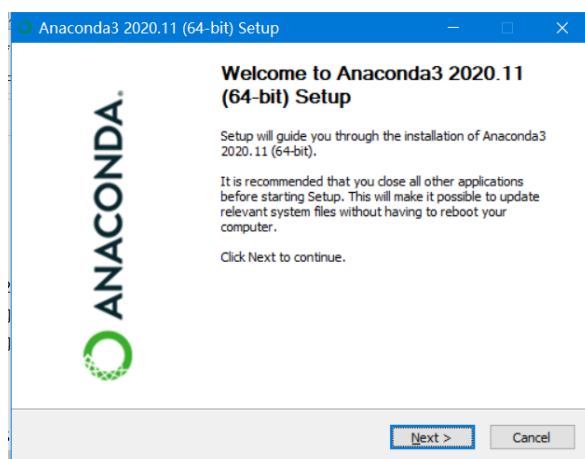
Windows			
Python version	Name	Size	SHA256 hash
Python 3.9	Miniconda3 Windows 64-bit	71.2 MiB	1acbc2e8277dd54a5f724896c7edee112d068529588d944702966c867e9ecc
Python 3.8	Miniconda3 Windows 64-bit	70.6 MiB	94f24e52e316fa935ccf94b0c504ceca8e6abc6190c68378e18550c95bb7cee1
Python 3.7	Miniconda3 Windows 64-bit	69.0 MiB	b221ccdb2bbc5a8209a292f858ae05fd87f882f79be75b37d26faa881523c057
Python 3.9	Miniconda3 Windows 32-bit	67.8 MiB	4fb64e6c9c28b88beab16994bfba4829110ea3145baa60bda5344174ab65d462
Python 3.8	Miniconda3 Windows 32-bit	66.8 MiB	60cc5874b3cce9d80a38fb2b28df96d880e8e95d1b5848b15c20f1181e2807db
Python 3.7	Miniconda3 Windows 32-bit	65.5 MiB	a6af674b984a333b53aaf99043f6af4f50b0bb2ab78e0b732aa60c47bbfb0704

(2) Anaconda 的安装

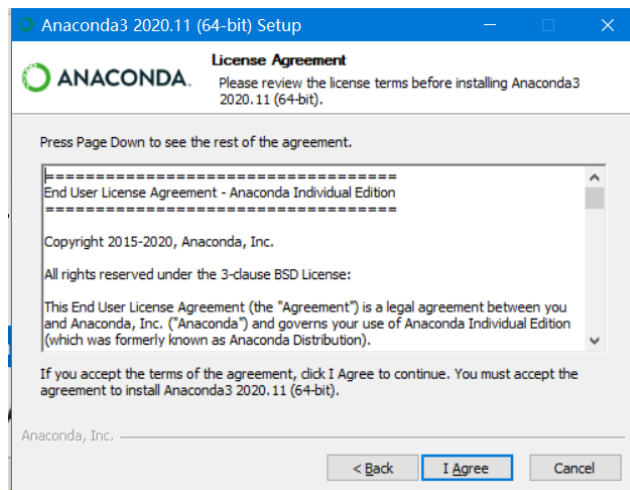
这里以 64 位的 Windows11 操作系统为例，介绍 anaconda 的安装及开发环境的建设，版本号为：[Anaconda3-2020.11-Windows-x86_64.exe](#)

其他操作系统下的 Anaconda 的安装及开发环境的建设，请读者参考其他教程。Anaconda 的安装步骤如下：

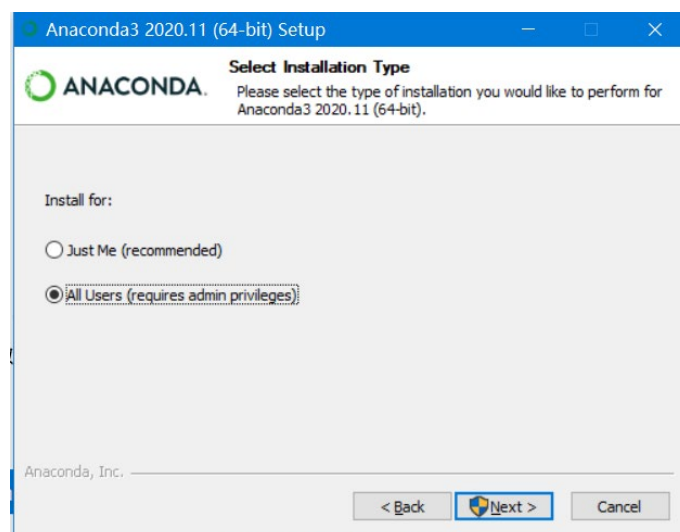
1) 双击即可下载 [Anaconda3-2020.11-Windows-x86_64.exe](#)，进入安装界面，如下图所示。



2) 单击“Next”按钮，进入“License Agreement”（安装协议）界面，单击“I Agree”按钮，如下图所示。



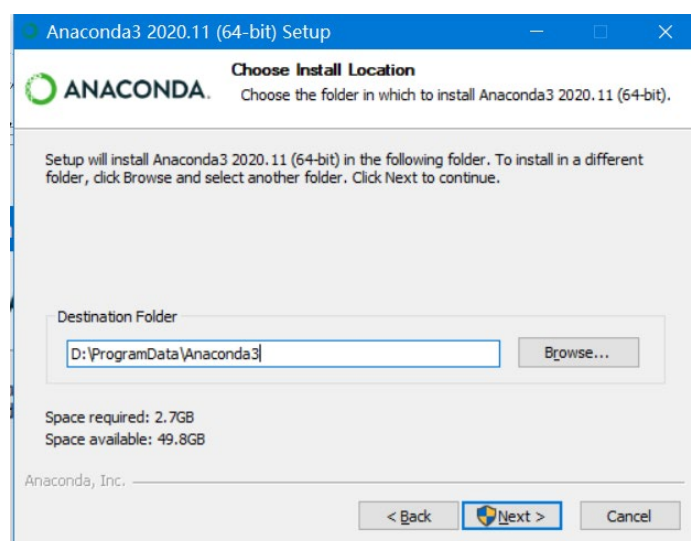
3) 出现权限使用界面, 如下图所示。如果是自己使用, 选中“Just Me”单选按钮; 如果允许其他人使用, 选中“All Users”单选按钮。然后单击“Next”按钮。



需要注意的是, 若 anaconda 的默认目录中事先安装有 anaconda 的早期版本, 或者说, 同名的 anaconda 文件夹不为空, 则无法进行安装。这时解决的方法通常有两个: 一是手动删除旧的安装目录, 保障目前 anaconda 安装路径的“纯洁性”; 而是选择不同的安装目录。

此外, 还需要注意的是, 安装路径**一定不能有空格或中文字符**, 因为 anaconda 暂时不支持间断性(含有空格)的安装路径和 Unicode 编码。

4) 进入下图所示的界面, 在这里选择安装路径。如果是读者自己使用且 C 盘有足够的空间, 可以安装在默认路径; 如果是使用非本人的计算机, 建议尽量不要安装在 C 盘, 因为可能发生权限问题。然后, 单击“Next”按钮。

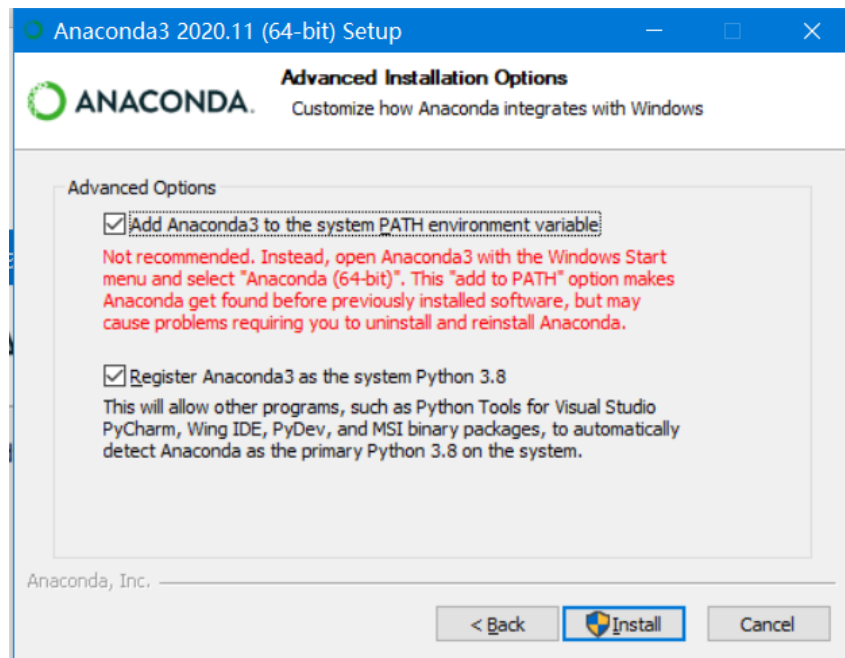


5) 进入下图所示界面, 这里可以选择自动修改 Windows 环境变量, 如果不选以后可以自行添加; 以及是否自动安装 Python 版本。选择完毕, 单击“Install”

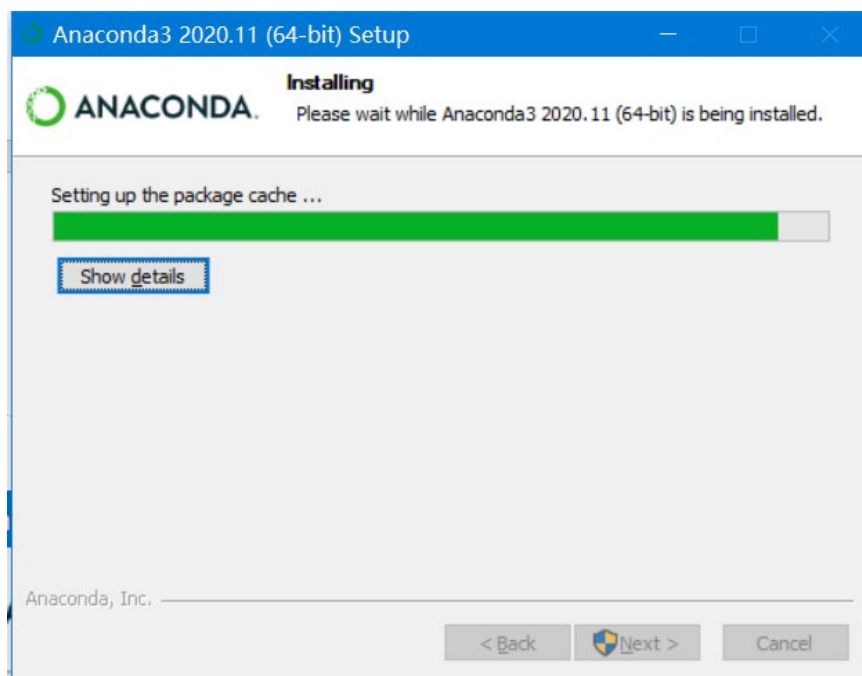
按钮。

【建议初学者将两个选项都选上。第一个选项，用户可以在任意命令行路径下启动 Python 或使用 conda 命令；

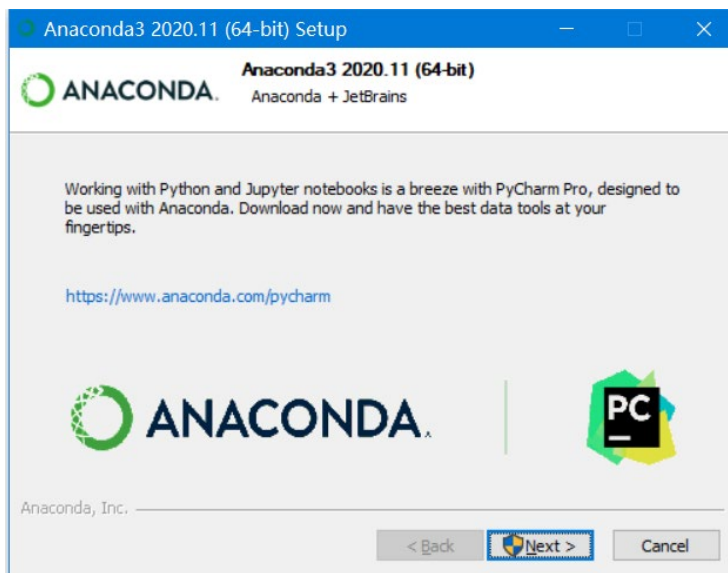
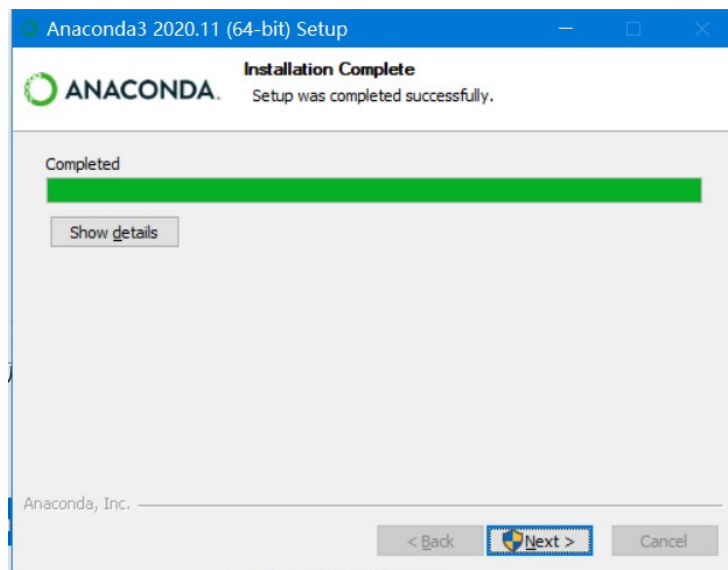
第二个选项选择 anaconda 作为默认的 Python 编译器，这个选项会令诸如 pycharm、Wing 等 IDE 开发环境自动检测 anaconda 的存在。】



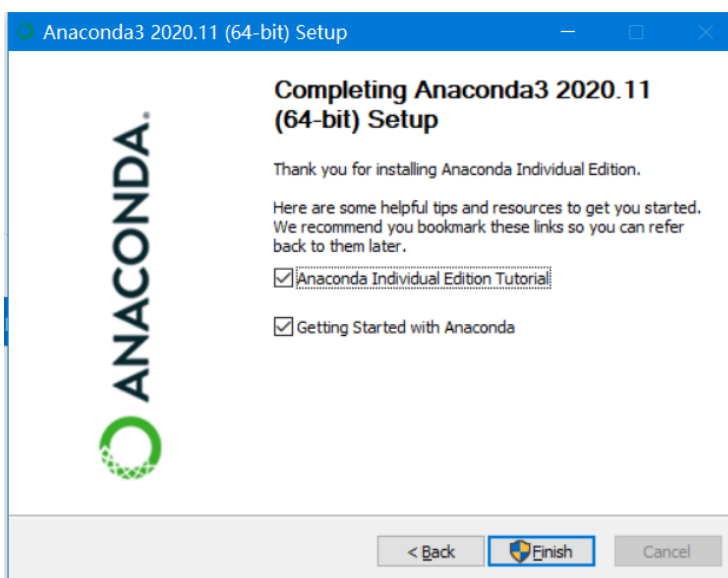
6) 进入下图所示安装界面，anaconda 开始自动安装。



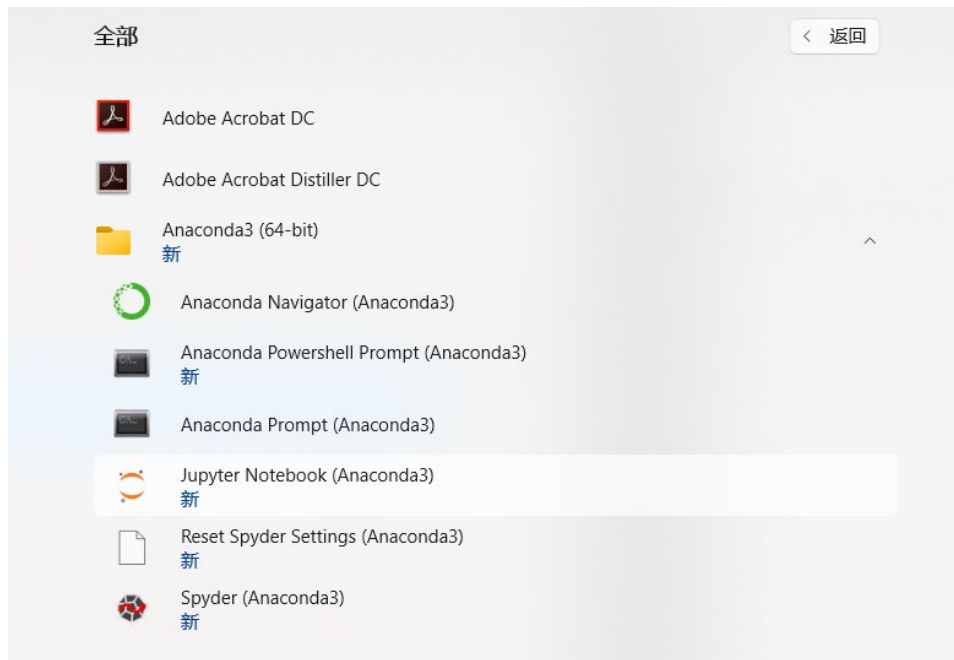
7) 当进度条进行至 Completed 时，安装完成，如图所示，单击“Next”按钮。若出现 VSCode 安装界面，可以选择不安装，如果后期需要课另行安装。单击“skip”按钮。



8) 进入安装完成界面，单击“Finish”按钮，安装结束。



9) 在开始这里就可以看到新添加的软件



10) 打开 Windows 终端（cmd）输入 python，验证 python 是否安装成功且版本型号为 python3.8.5。

```

管理员: C:\Windows\System32\cmd.exe - python
Microsoft Windows [版本 10.0.19044.1379]
(c) Microsoft Corporation. 保留所有权利。

C:\Windows\System32>python
Python 3.8.5 (default, Sep 3 2020, 21:29:08) [MSC v.1916 64 bit (AMD64)] :: Anaconda, Inc. on win32

Warning:
This Python interpreter is in a conda environment, but the environment has
not been activated. Libraries may fail to load. To activate this environment
please see https://conda.io/activation

Type "help", "copyright", "credits" or "license" for more information.
>>>

```

11) 以管理员身份运行 Anaconda Prompt (Anaconda) 并在命令行输入 conda list 命令，若安装成功则会显示安装盘符和版本信息。

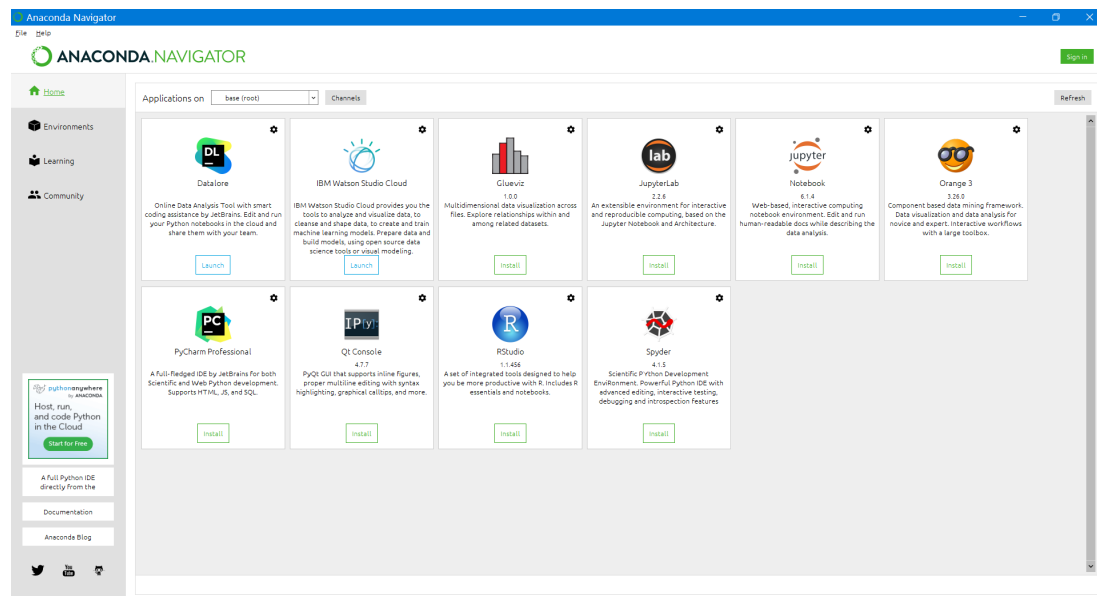
```

管理员: Anaconda Prompt (Anaconda3)

(base) C:\Users\Administrator>conda list
# packages in environment at D:\ProgramData\Anaconda3:
#
# Name                   Version           Build    Channel
_ipyw_jlab_nb_ext_conf  0.1.0             py38_0
alabaster                0.7.12            py_0
anaconda                 2020.11           py38_0
anaconda-client          1.7.2             py38_0
anaconda-navigator       1.10.0            py38_0
anaconda-project         0.8.4             py_0
argh                     0.26.2            py38_0
argon2-cffi              20.1.0            py38he774522_1
asn1crypto               1.4.0             py_0
astroid                  2.4.2             py38_0
astropy                  4.0.2             py38he774522_0
async_generator           1.10              py_0
atomicwrites             1.4.0             py_0
attrs                   20.3.0            pyhd3eb1b0_0
autopep8                 1.5.4             py_0
babel                    2.8.1             pyhd3eb1b0_0
backcall                 0.2.0             py_0
backports                1.0               py_2
backports.functools_lru_cache 1.6.1            py_0
backports.shutil_get_terminal_size 1.0.0           py38_2
backports.tempfile       1.0               py_1
backports.weakref        1.0.post1         py_1
bcrypt                   3.2.0             py38he774522_0
beautifulsoup4           4.9.3             pyhb0f4dca_0
bitarray                 1.6.1             py38h2bbff1b_0

```

或者直接启动 Anaconda Navigator (Anaconda)，若能成功启动也说明 Anaconda 安装成功了。



2、Anaconda 的基本使用

Anaconda 个人版本中包含 Conda 和 Anaconda Navigator。Conda 适用于你的命令行界面，也就是 Windows 上的 Anaconda Prompt (Anaconda) 和 macOS 和 Linux 上的终端。Navigator 是一个桌面图形用户界面，允许在不使用命令行命令的情况下启动应用程序并轻松管理 Conda 包、环境和通道。我们可以同时尝试使用 Conda 和 Navigator，看看哪个适合你管理你的包和环境，甚至可以在它们之间切换，并且可以在另一个中查看你对其中一个所做的工作。此外，这里还简单介绍了 Spyder 和 Jupyter Notebook 运行 python 程序的相关操作。

(1) Anaconda Prompt

Anaconda Prompt 集成了最常用的 Conda 命令集，用户可以直接使用 conda 命令。用命令“conda list”查看已安装的包，从这些库当中可以发现 Numpy、Scipy、Matplotlib 和 Pandas，说明已经安装成功了。

在 Anaconda Prompt (Anaconda) 中可以通过键入命令来进行相应的操作。下面的三条命令有着相同的功能，可以查看 conda 中已有的环境信息。

`conda info -e`

`conda env list`

`conda info --envs`


```
管理员: Anaconda Prompt (Anaconda3)
zict                2.0.0                py_0
zipp                3.4.0                pyhd3eb1b0_0
zlib                1.2.11             h62dc497_4
zope                1.0                py38_1
zope.event          4.5.0                py38_0
zope.interface      5.1.2             py38he774522_0
zstd                1.4.5             h04227a9_0

(base) C:\Users\Administrator>conda --version
conda 4.9.2

(base) C:\Users\Administrator>conda info -e
# conda environments:
#
base                  * D:\ProgramData\Anaconda3

(base) C:\Users\Administrator>conda env list
# conda environments:
#
base                  * D:\ProgramData\Anaconda3

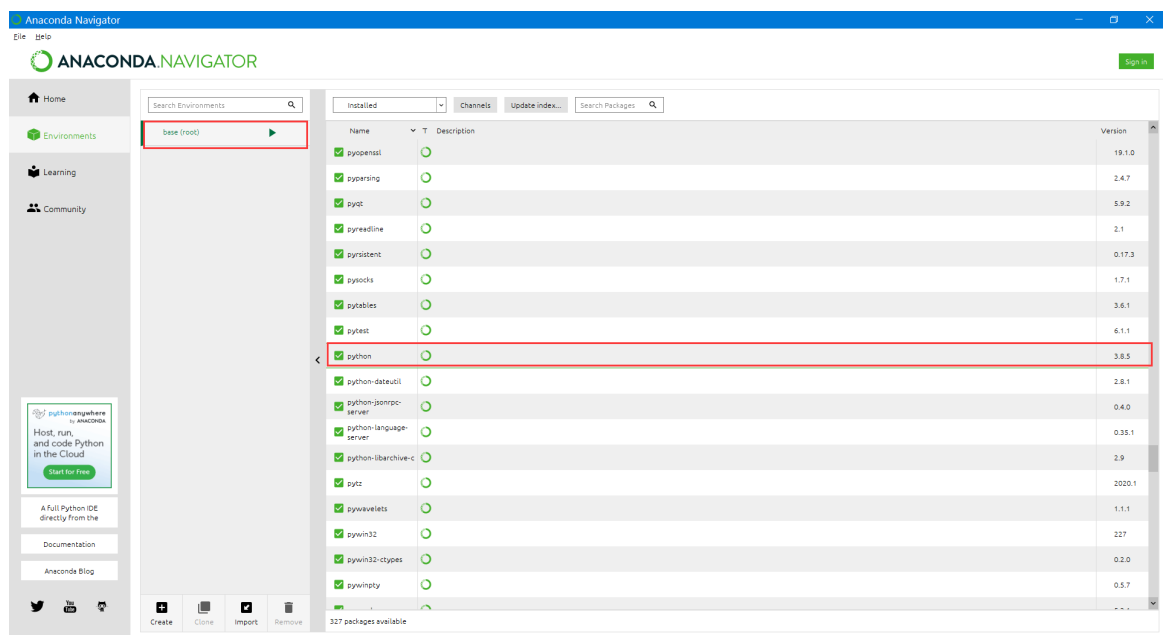
(base) C:\Users\Administrator>conda info --envs
# conda environments:
#
base                  * D:\ProgramData\Anaconda3

(base) C:\Users\Administrator>
```

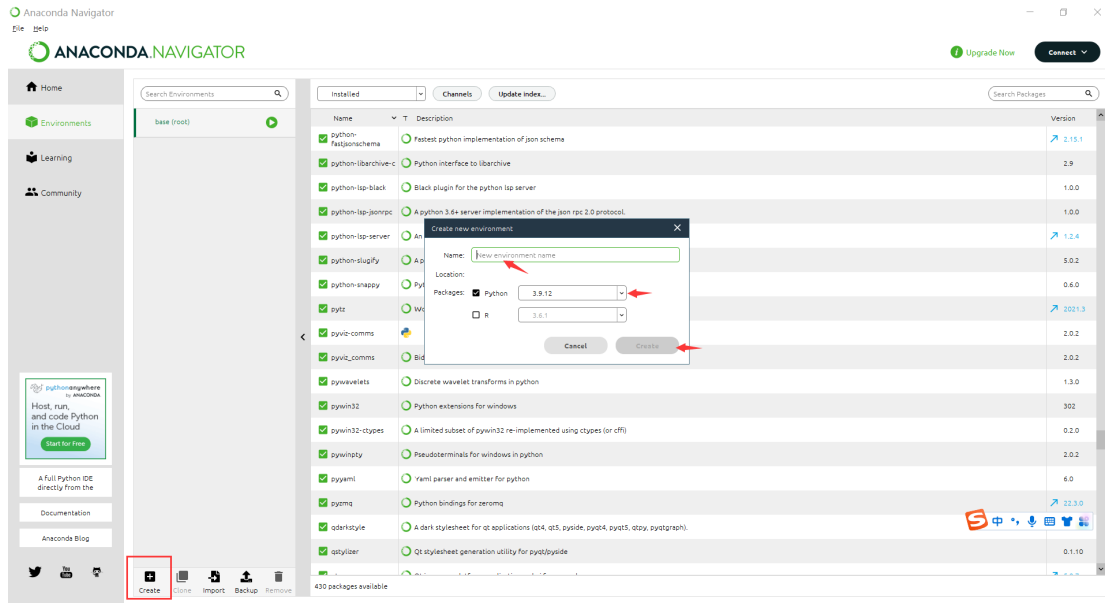
(2) Anaconda Navigator

Anaconda Navigator 是用于管理工具包和环境的图形用户界面，后续涉及的众多管理命令也可以在 Anaconda Navigator 中手动实现。

打开 Anaconda Navigator (Anaconda) 后其界面如下图所示，环境中只有一个名为 base(root) 的环境，查看 python 版本信息为 3.8.5，这与我们安装 Anaconda 时勾选的版本是一致的。



如果要添加一个新的环境，点击 Create，在弹出的对话框中输入环境名称，生成即可，可以看到其路径在 Anaconda 的安装路径下。



可以看到，现在就有两个环境了，如果你想激活哪一个，就鼠标单击它，后面有小三角的就是被激活的那一个。

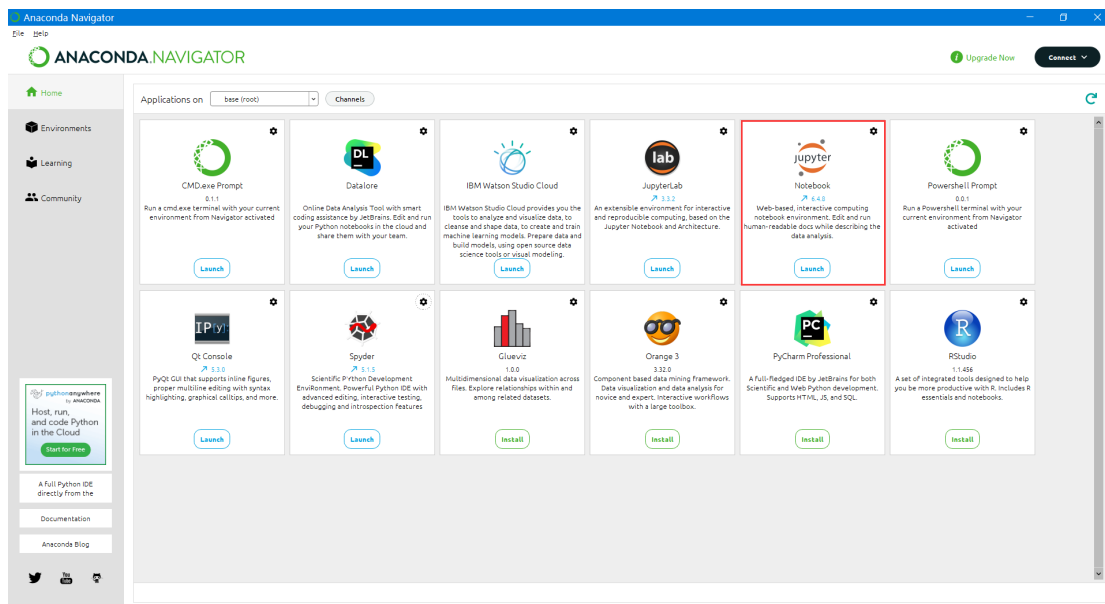
这时候再通过 Anaconda Prompt (Anaconda) 查看环境信息如下图所示。

这就是前面介绍中所说的可以在另一个中查看你对其中一个所做的工作。

在 Anaconda Navigator (Anaconda) 中还可以将还未安装的添加到当前环境。】

(3) Jupyter Notebook

Jupyter Notebook 是基于 Web 的交互式计算环境，可以编辑易于人们阅读的文档，用于展示数据分析的过程。



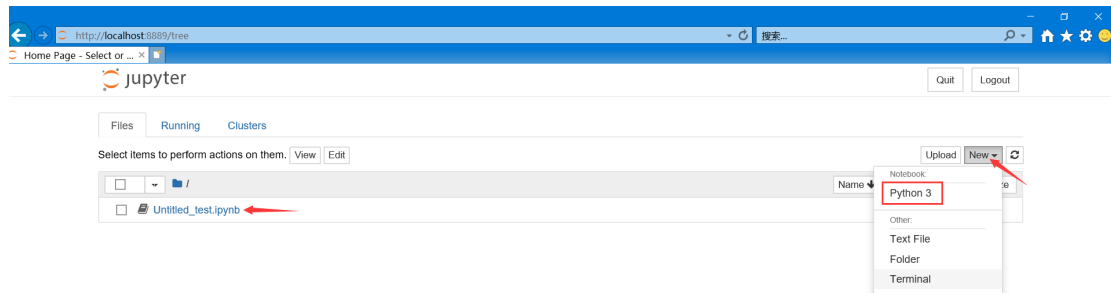
单击“Launch”按钮会在默认浏览器打开 <http://localhost:8888/tree> 这样的页面，如图所示。单击 Notebook 下面的“Python3”图标就可以运行 Python，这里

输入 `print('hello World')`,然后单击  按钮运行程序。

在命令窗口中输入“Jupyter Notebook”也可直接打开 Jupyter Notebook。

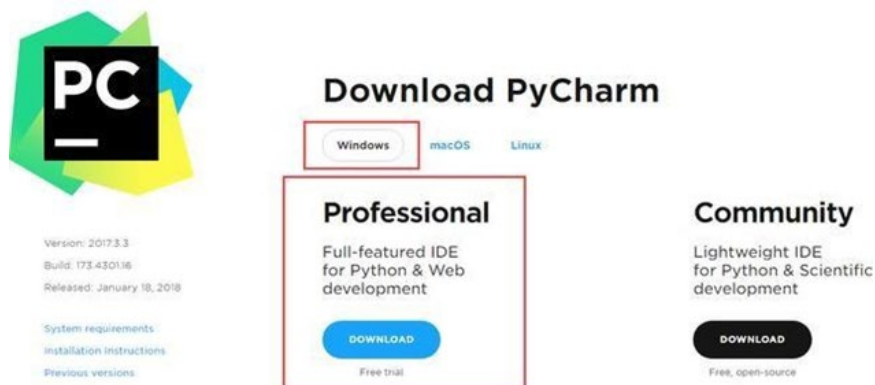
点击 New, 选择你安装的 Python 版本（我这里是 Python 3），然后会弹出一个新的网页，在第一行键入 python 代码。

你可以双击顶部 Untitled 字样重命名该文件名，如下图所示。



3、下载并安装 PyCharm 编辑器

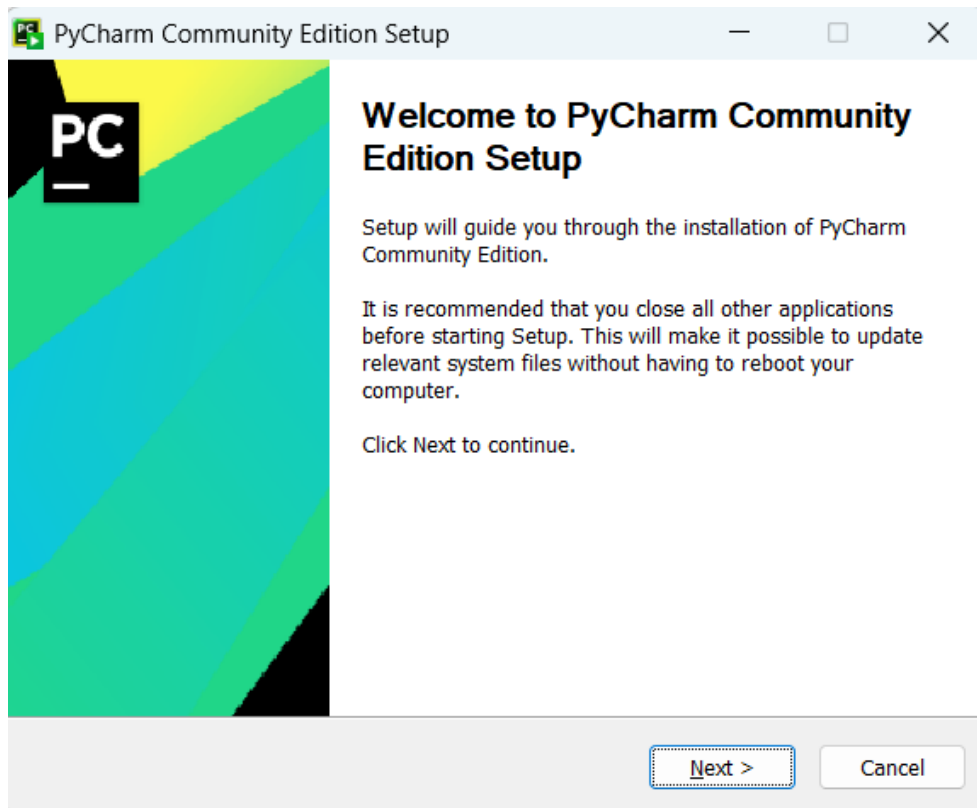
这里建议安装学习通上次版本软件，也可以去 Pycharm 官网，或者直接输入网址：<http://www.jetbrains.com/pycharm/download/#section=windows>，下载 PyCharm 安装包，根据自己电脑的操作系统进行选择，对于 windows 系统选择下图的框框所包含的安装包。



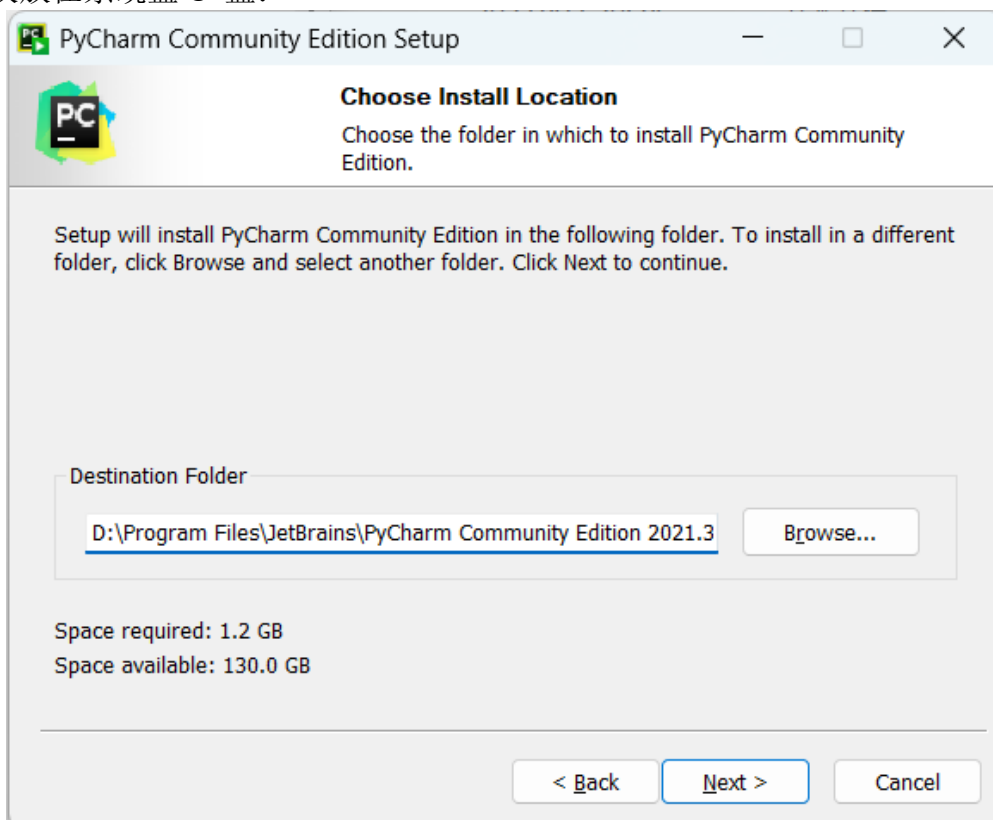
选择 Windows 系统的专业版，将其下载到本地，如下图所示：

	pycharm-professional-2016.1.3.exe	2017/4/17 15:17	应用程序	215,559 KB
	pycharm-professional-2017.1.1.exe	2017/4/21 10:09	应用程序	208,358 KB
	pycharm-professional-2017.2.2.exe	2017/8/30 22:31	应用程序	215,750 KB
	python-2.7.12.msi	2017/4/17 15:22	Windows Instal...	18,464 KB
	python-3.4.3 32 64都可以.msi	2017/4/18 11:01	Windows Instal...	24,264 KB

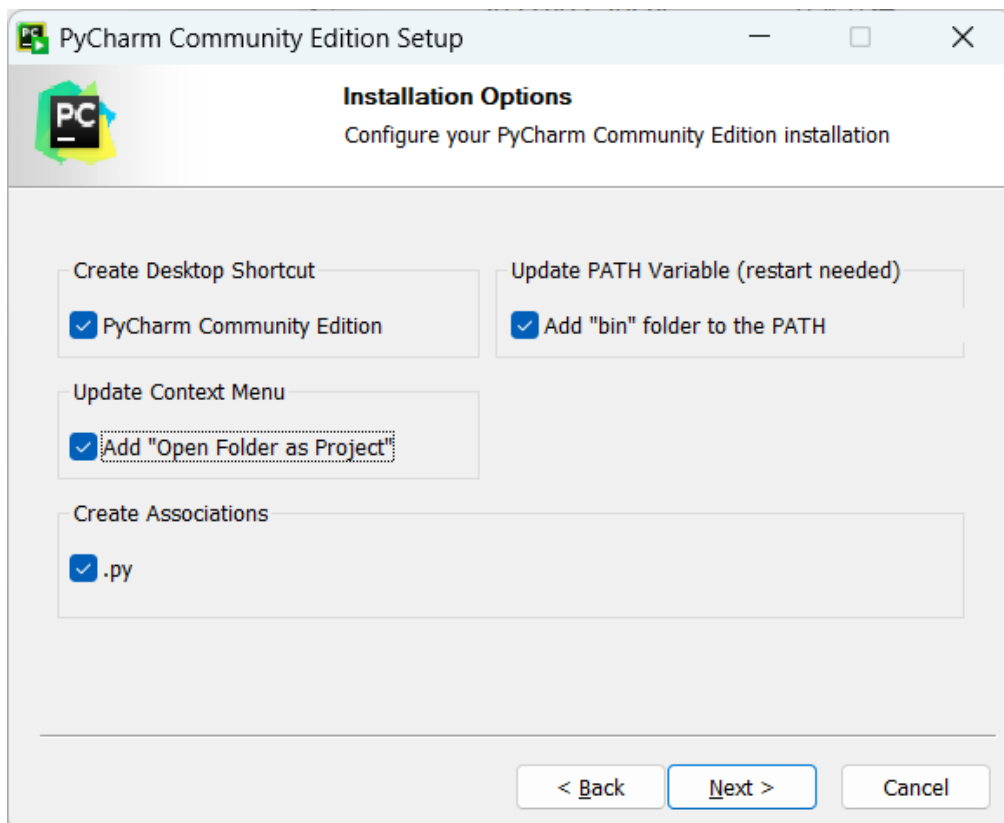
双击下载的安装包，进行安装，然后会弹出界面：



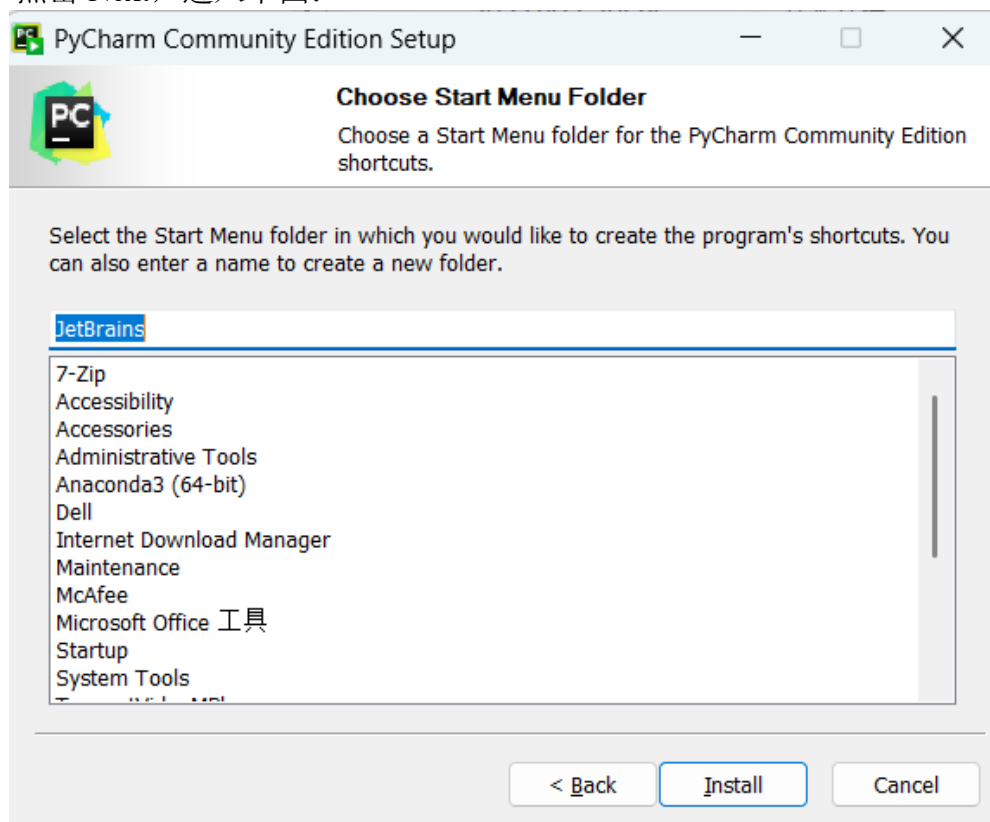
选择安装目录, Pycharm 需要的内存较多, 建议将其安装在 D 盘或者 E 盘, 不建议放在系统盘 C 盘:



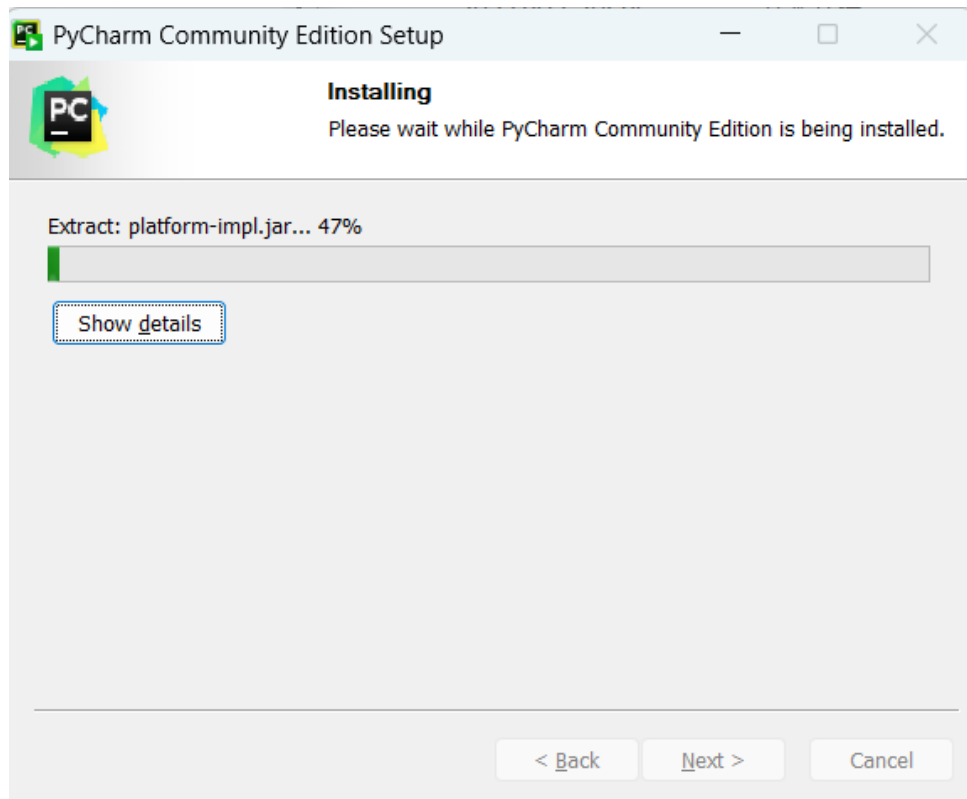
点击 Next, 进入下图的界面, 所有选项打对勾, 选项分别为: 创建桌面快捷方式、更新菜单、创建文件关联、更新 PATH 变量



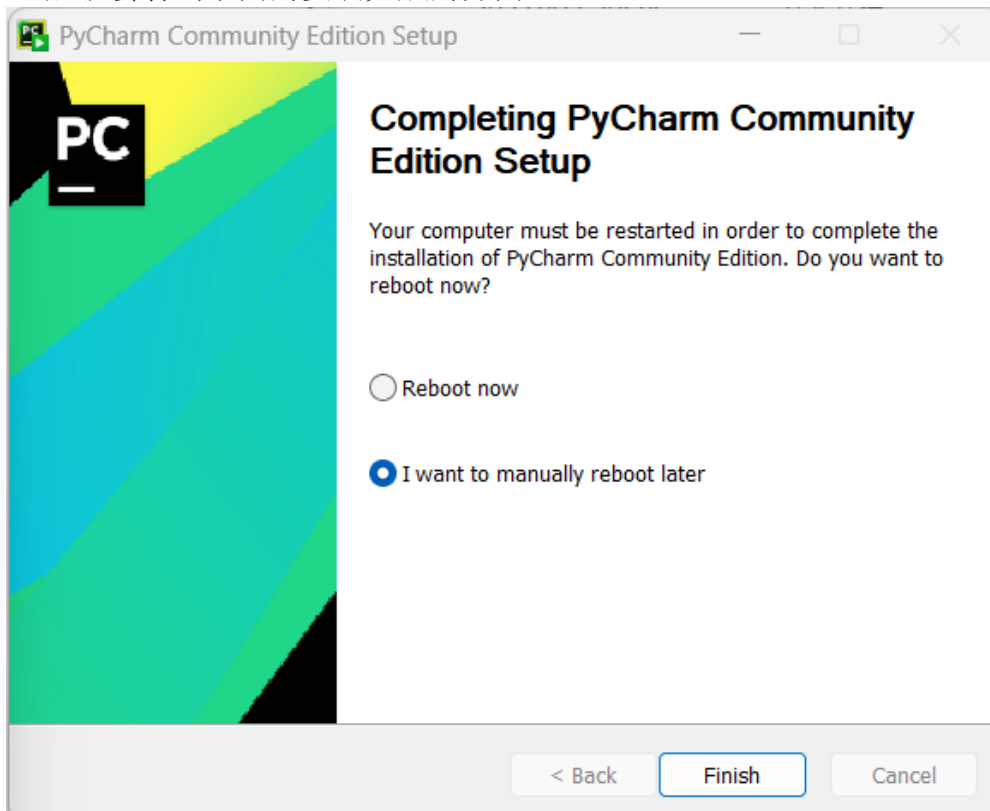
点击 Next，进入下图：



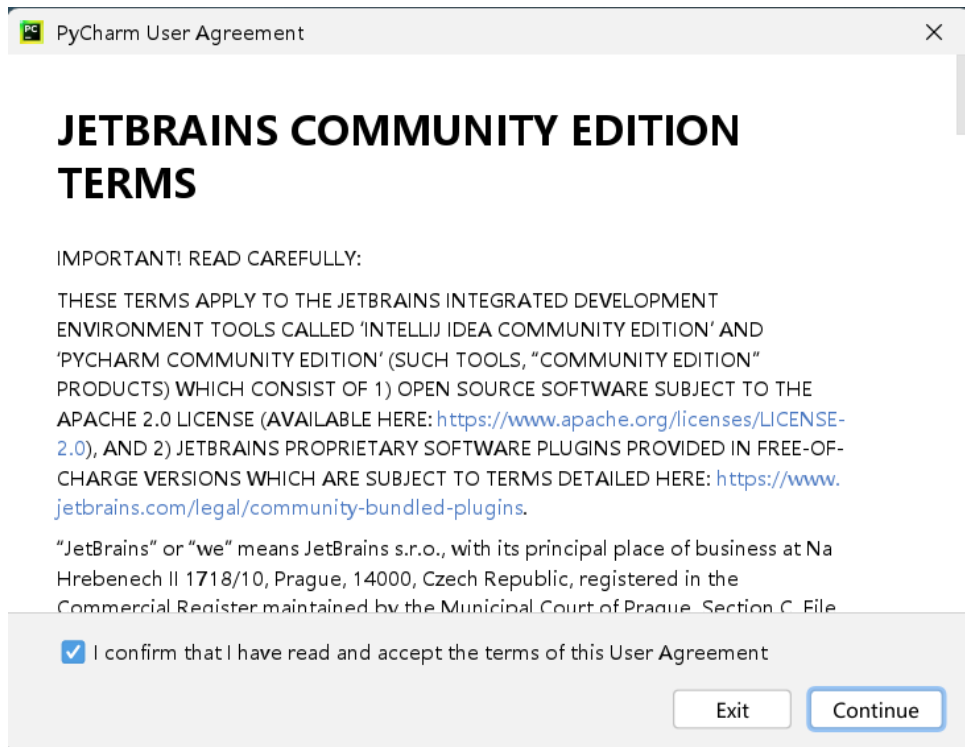
默认安装即可，直接点击 Install。耐心的等待两分钟左右，如下图：



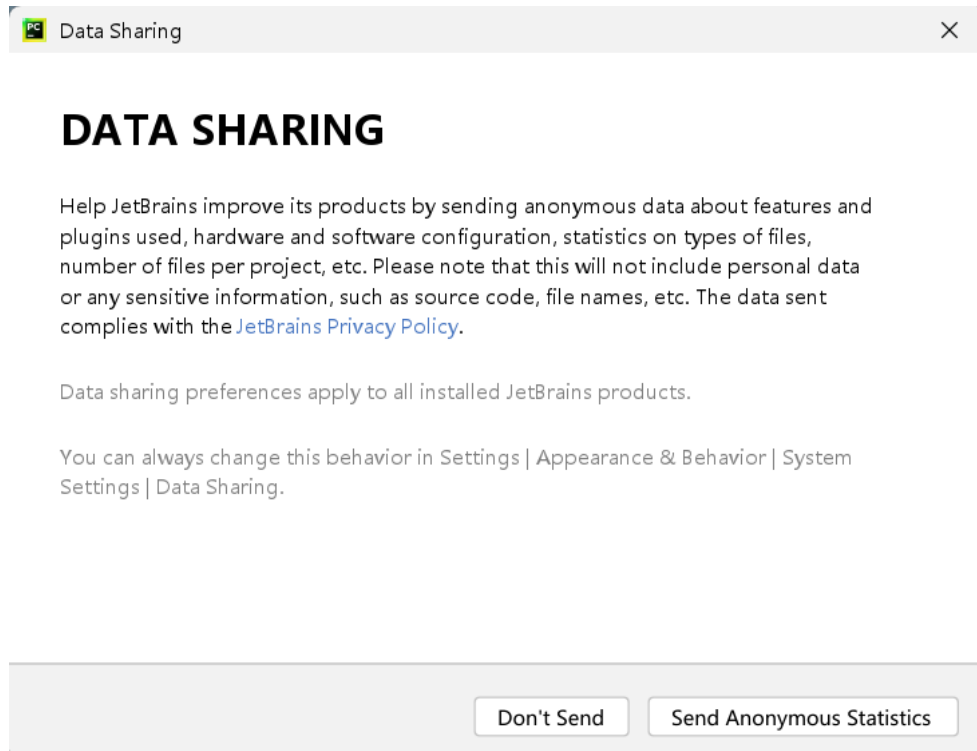
之后就会看到下面的安装完成的界面：



选择，点击 I want to manually reboot later 我想稍后手动重启，然后点击 Finish，Pycharm 安装完成。接下来对 Pycharm 进行配置，双击运行桌面上的 Pycharm 图标，进入下图界面：



同意打对钩后，点击 Continue，如下图所示：

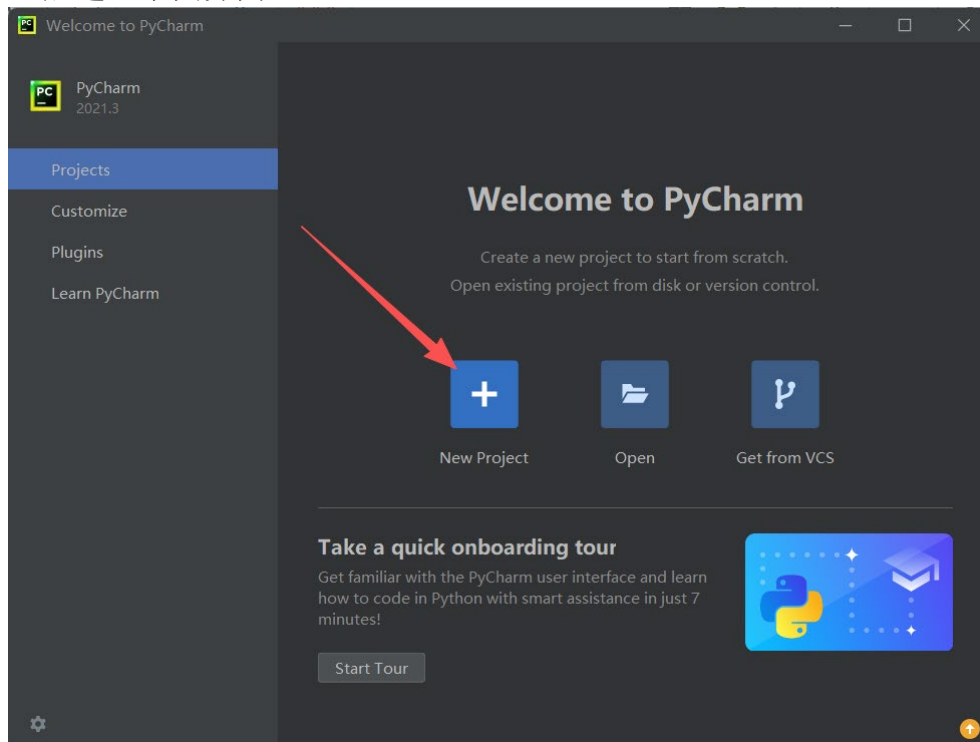


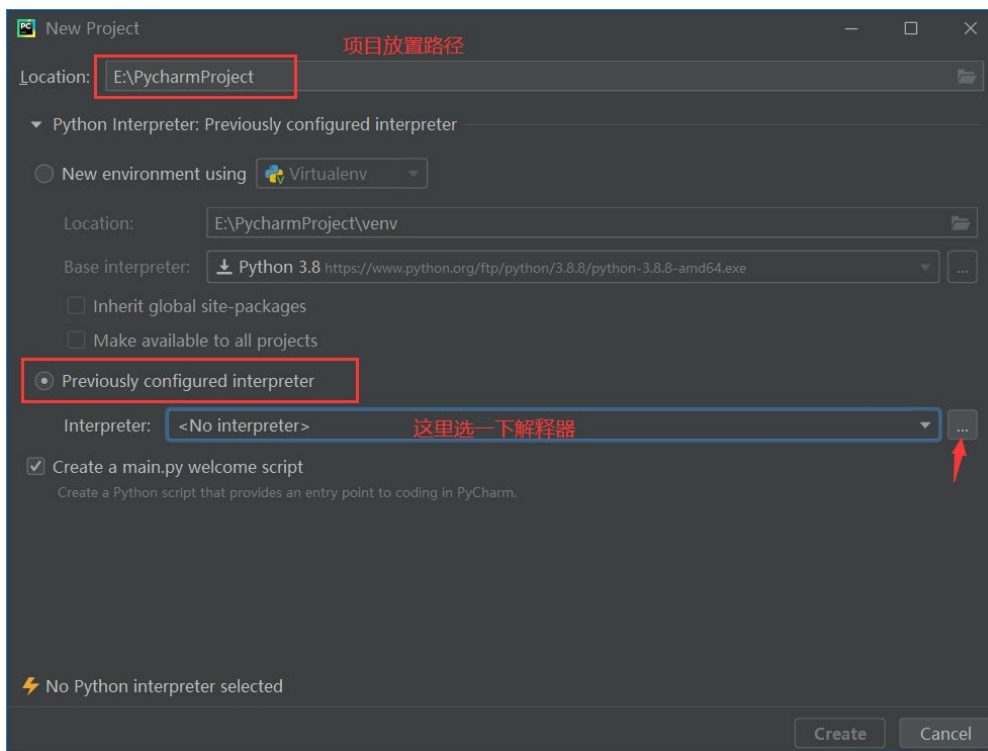
PyCharm 分享的是一些功能和插件、硬件和软件配置等信息不包括源码，个人数据等一些重要数据。这里我们选择不分享即点击上图中的标注 1 Don't Send 按钮。

- ①： Don't Send（不要发送，不分享数据）
- ②： Send Anonymous Statistics（发送匿名统计信息，匿名分享数据）

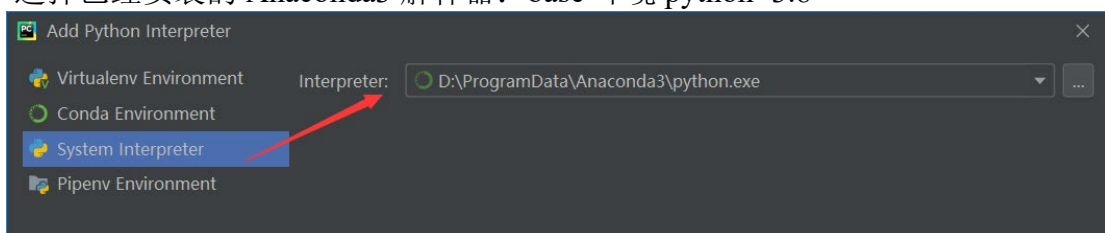


之后进入下图界面：

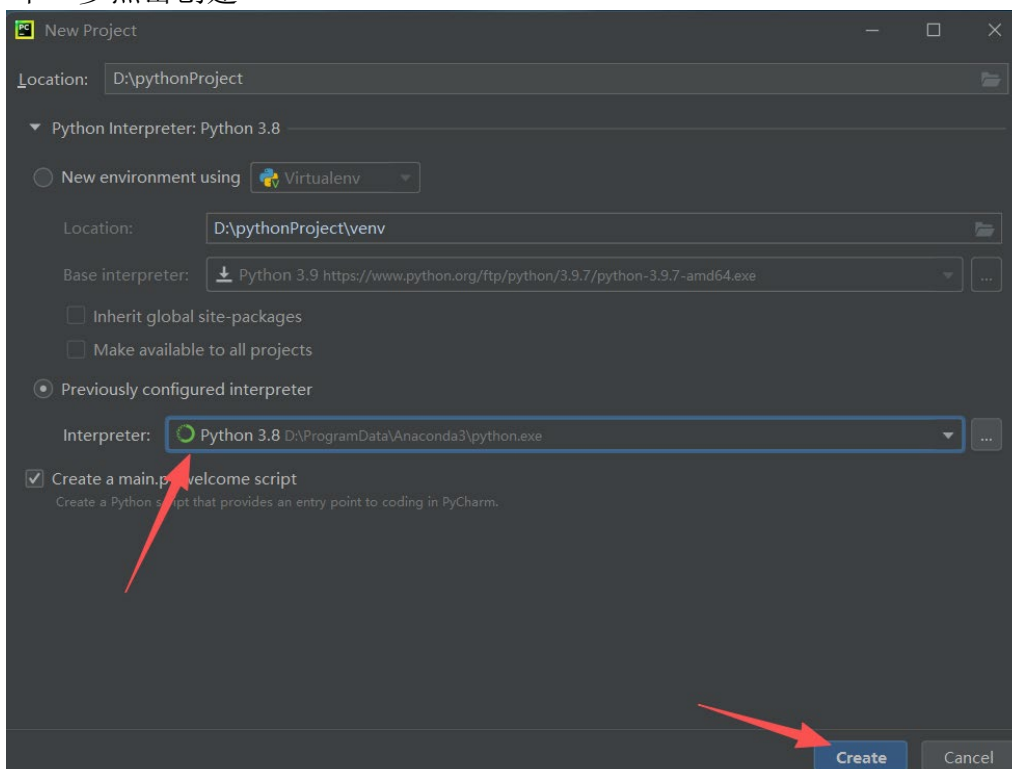




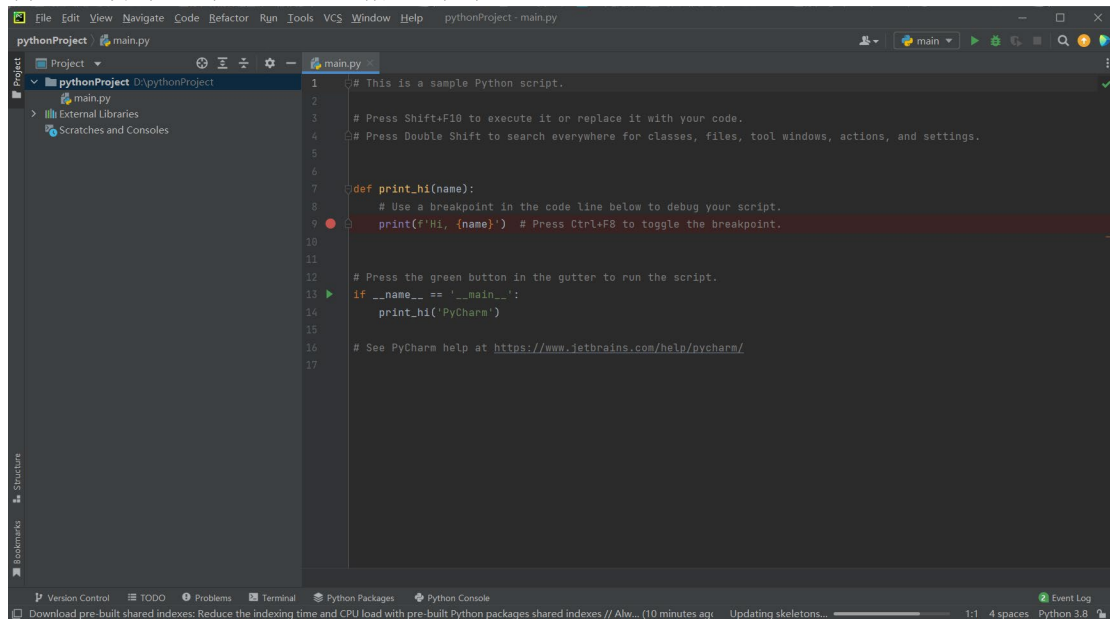
选择已经安装的 Anaconda3 解释器: base 环境 python=3.8



下一步点击创建



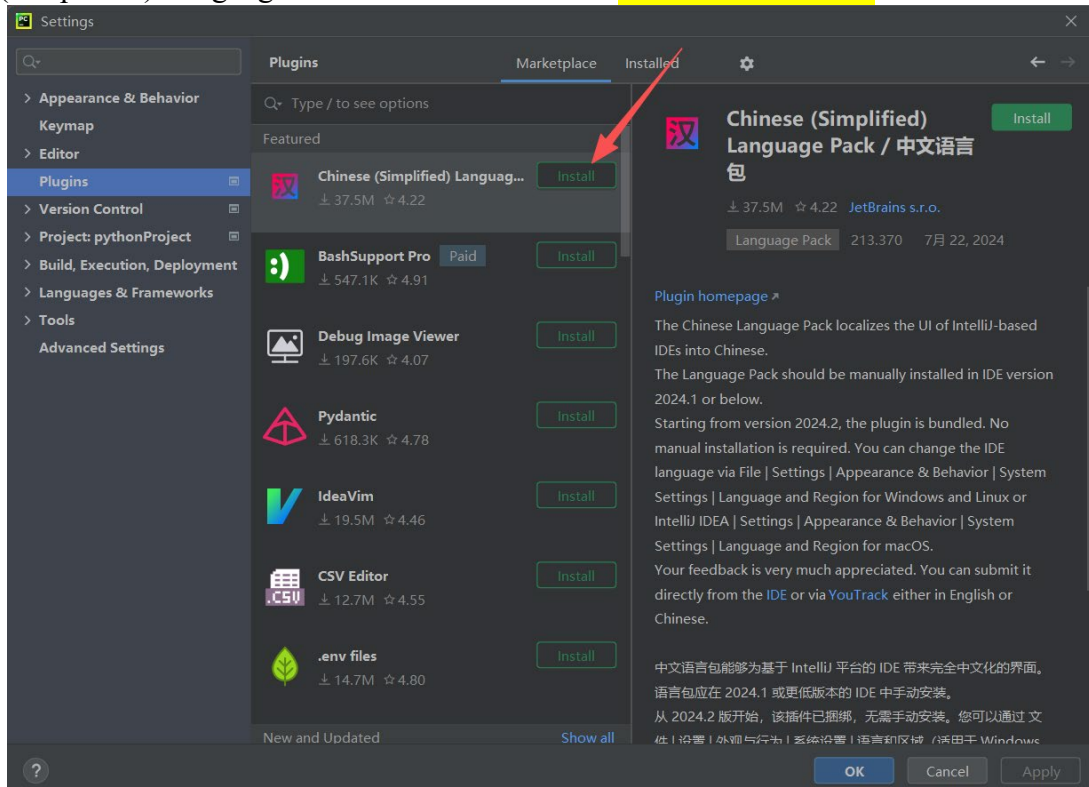
这个界面是 Pycharm 默认的界面,大家完全可以自己去 setting 中设置自己喜欢的背景和字体大小、格式等等。



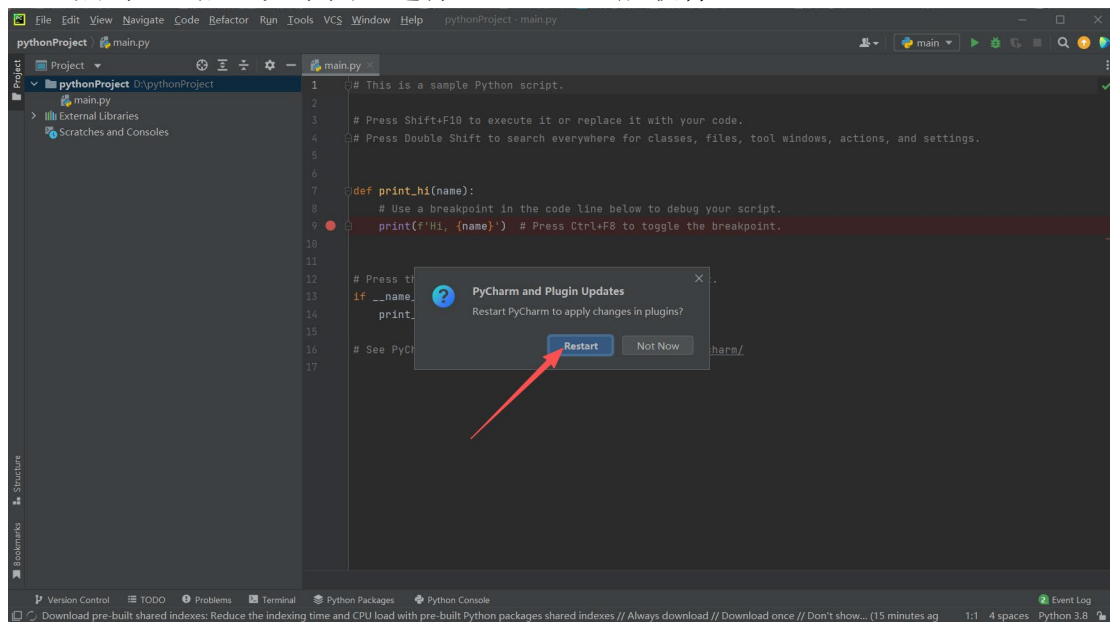
Pycharm 中的部分快捷键:

- Ctrl + Enter: 在下方新建行但不移动光标;
- Shift + Enter: 在下方新建行并移到新行行首;
- Ctrl + /: 注释(取消注释)选择的行;
- Ctrl+d: 对光标所在行的代码进行复制。

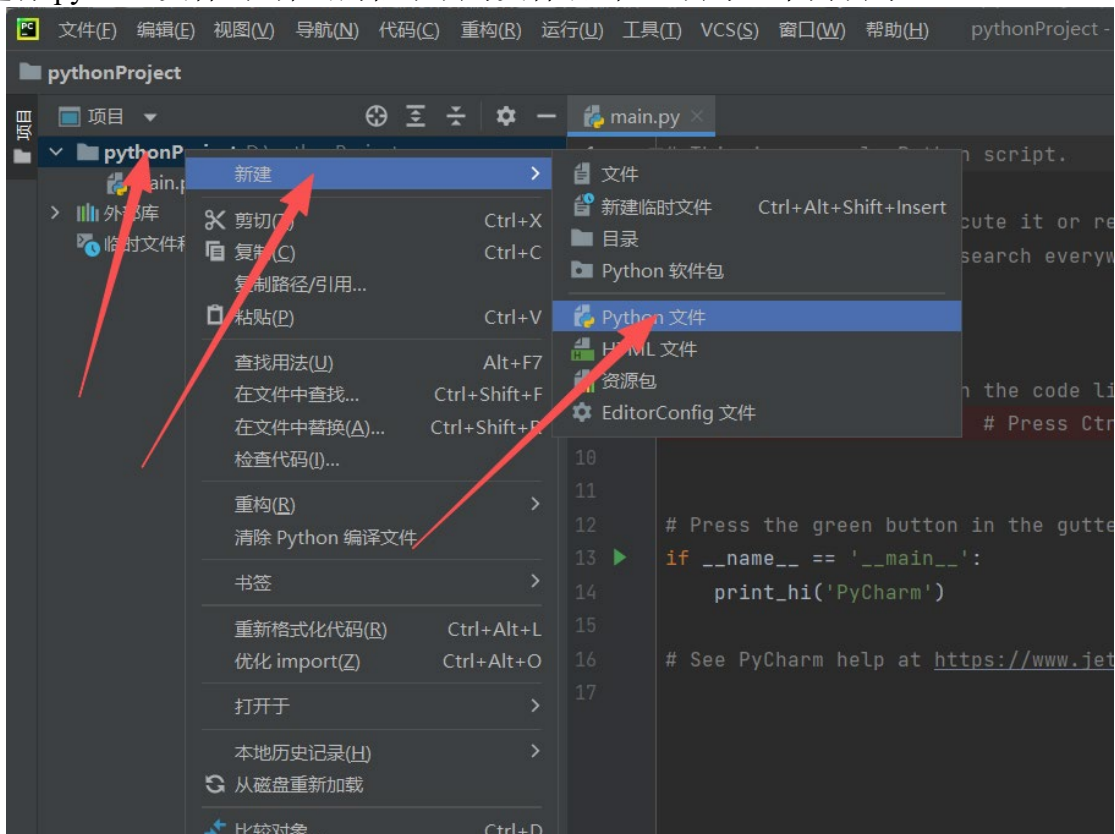
点击左侧的 File,再点击菜单中的 Settings 选项,找到 Plugins, 找到 Chinese (Simplified) Language...点击 Install, 进行汉化(根据个人需要)



点击 OK 后，如下图，选择 Restart，重启软件



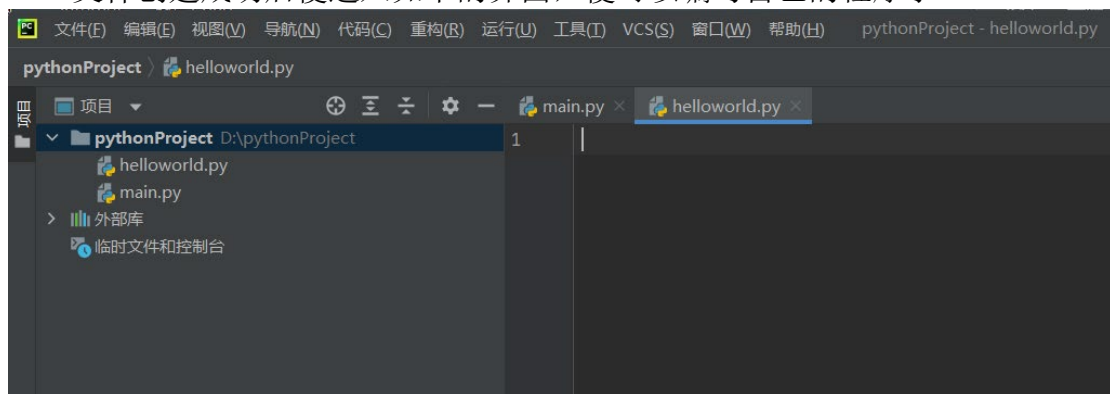
重启后软件已汉化，鼠标右击图中箭头指向的地方，然后选择“新建”，最后选择 python 文件，在弹出的框中填写文件名(任意填写)，本例填写: helloworld。



之后得到下图，然后 双击 或者 回车 即可：

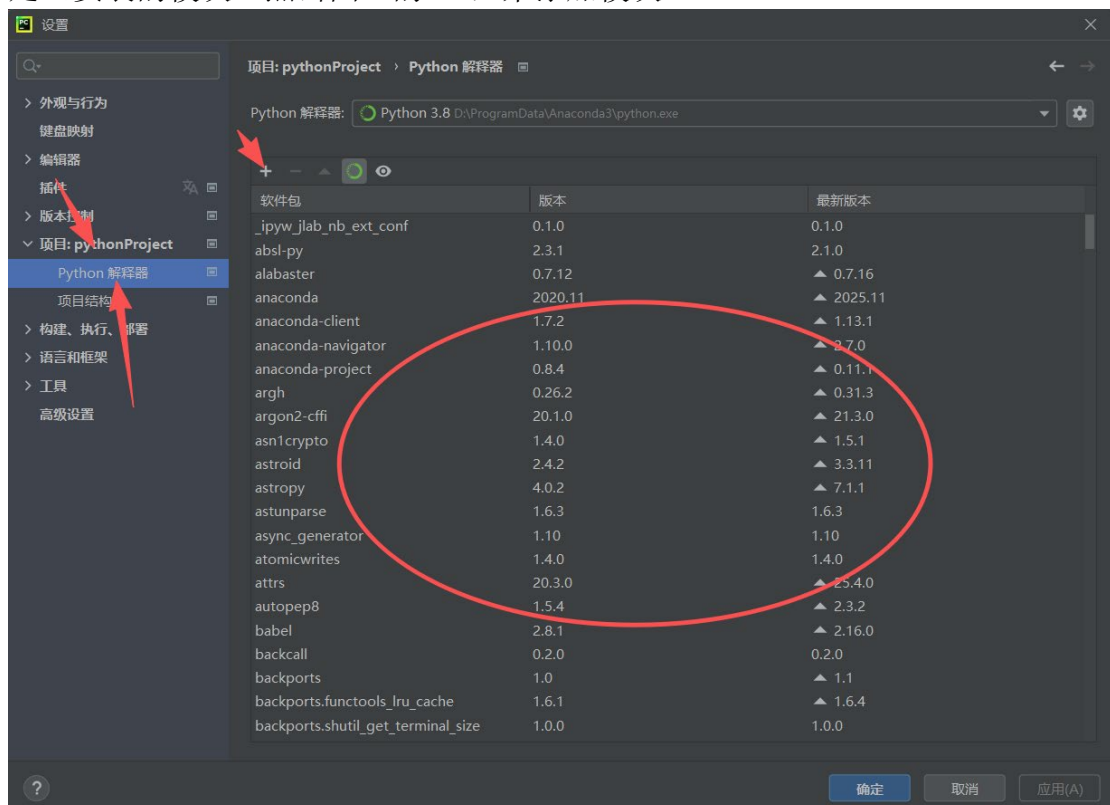


文件创建成功后便进入如下的界面，便可以编写自己的程序了。

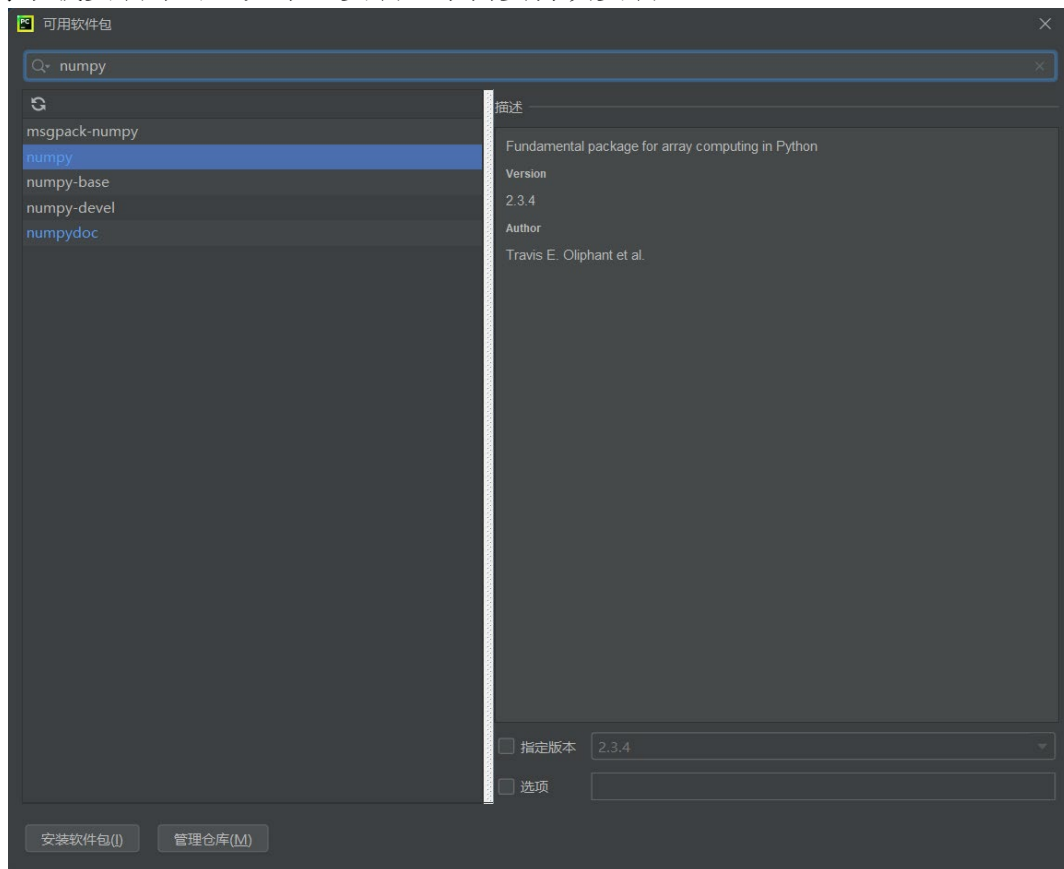


Python 中安装第三方库

打开 Pycharm, 点击左侧的 文件, 再点击菜单中的 设置 选项, 在弹出的“设置”菜单栏中, 找到自己的项目, 点击下面的“Python 解释器”, 在右侧显示的就是已安装的模块。点击图上的“+”, 来添加模块。

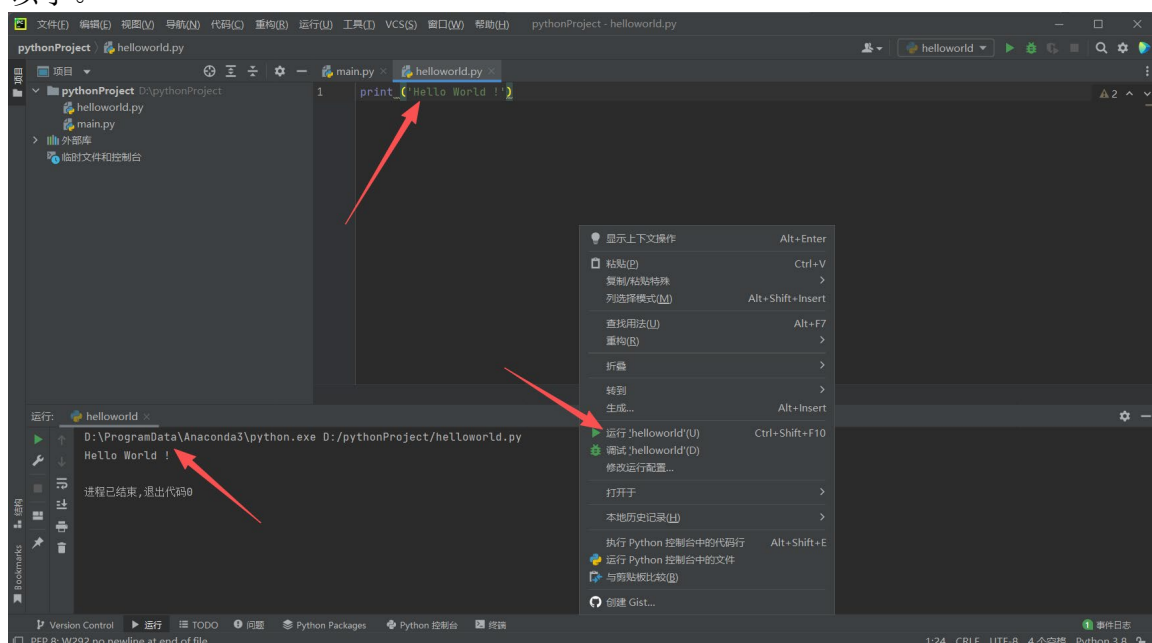


在图上侧的搜索框输入“numpy”,左侧就会搜索到对应的模块,点击后,右侧显示模块的版本和详细信息。如果未安装,再点击最下边的“Install Package”,等待下载安装即可。如果已安装,不需要再次安装。



回到刚才新建的文件 Hello World

编写第一个 Python 程序。输入 `print('Hello World !')` 后,右键运行代码就可以了。



4、简单项目练习

(1) 猜数字游戏

```
import random

def guess_number():
    """
    猜数字游戏
    电脑随机生成一个 1-100 的数字，玩家有 7 次机会猜测
    """
    number = random.randint(1, 100)
    attempts = 7

    print("欢迎来到猜数字游戏！")
    print("我已经想了一个 1 到 100 之间的数字，你有 7 次机会来猜。")

    for attempt in range(1, attempts + 1):
        try:
            guess = int(input(f"\n 第 {attempt} 次尝试，请输入你的猜测: "))
        except ValueError:
            print("请输入一个有效的整数！")
            continue

        if guess < number:
            print("太小了！")
        elif guess > number:
            print("太大了！")
        else:
            print(f"恭喜你！你在第 {attempt} 次猜对了！")
            break

    else:
        print(f"\n 很遗憾，你没有猜对。正确答案是: {number}")

# 运行游戏
guess_number()
```

运行程序，猜数字

(2) 简易计算器

```
def calculator():
    """
    简易计算器
    支持加、减、乘、除四种基本运算
    """
    print("简易计算器")
    print("支持的操作: +, -, *, /")

    while True:
        try:
            num1 = float(input("请输入第一个数字: "))
            operator = input("请输入运算符 (+, -, *, /): ")
            num2 = float(input("请输入第二个数字: "))

            if operator == '+':
                result = num1 + num2
            elif operator == '-':
                result = num1 - num2
            elif operator == '*':
                result = num1 * num2
            elif operator == '/':
                if num2 == 0:
                    print("错误: 除数不能为零! ")
                    continue
                result = num1 / num2
            else:
                print("不支持的运算符! ")
                continue

            print(f"结果: {num1} {operator} {num2} = {result}")

        except ValueError:
            print("请输入有效的数字! ")

    # 询问是否继续
    again = input("\n 是否继续计算? (y/n): ").lower()
```



```
if again != 'y':  
    print("感谢使用计算器！")  
    break
```

```
# 运行计算器  
calculator()
```

运行程序，做简单计算

（二）约会网站配对效果判定（24 页-49 页）

1、k-近邻法简介

k 近邻法(k-nearest neighbor, k-NN)是 1967 年由 Cover T 和 Hart P 提出的一种基本分类与回归方法。它的工作原理是：存在一个样本数据集合，也称之为训练样本集，并且样本集中每个数据都存在标记，即我们知道样本集中每一个数据与所属分类的对应关系。输入没有标记的新数据后，将新的数据的每个特征与样本集中数据对应的特征进行比较，然后算法提取样本最相似数据(最近邻)的分类标记。

一般来说，我们只选择样本数据集中前 k 个最相似的数据，这就是 k-近邻算法中 k 的出处，通常 k 是不大于 20 的奇数。最后，选择 k 个最相似数据中出现次数最多的分类，作为新数据的分类。

那么，用什么方法进行判断哪些数据是最近邻呢？没错，就是距离度量。如果数据有 2 个特征，也就是在 2 维实数向量空间，可以使用我们高中学过的两点距离公式计算距离。

$$|AB| = \sqrt{(x_1 - x_2)^2 + (y_1 - y_2)^2}$$

但是如果是更高维的呢？对，没错。我们可以用欧氏距离(也称欧几里德度量)，也可写成如下公式：

$$\begin{aligned} d(p, q) &= d(p, q) = \sqrt{(q_1 - p_1)^2 + (q_2 - p_2)^2 + \dots + (q_n - p_n)^2} \\ &= \sqrt{\sum_{i=1}^n (q_i - p_i)^2} \end{aligned}$$

我们高中所学的两点距离公式就是欧氏距离在二维空间上的公式，也就是欧氏距离的 n 的值为 2 的情况。

我们总结 k-近邻算法的一般流程为：

收集数据：可以使用爬虫进行数据的收集，也可以使用第三方提供的免费或收费的数据。一般来讲，数据放在 txt 文本文件中，按照一定的格式进行存储，便于解析及处理。

准备数据：使用 Python 解析、预处理数据。

分析数据：可以使用很多方法对数据进行分析，例如使用 Matplotlib 将数据可视化。

测试算法：计算错误率。

使用算法：错误率在可接受范围内，就可以运行 k-近邻算法进行分类。

已经了解了 k-近邻算法的一般流程，下面开始进入实验项目内容。

2、项目练习

海伦女士一直使用在线约会网站寻找适合自己的约会对象。尽管约会网站会推荐不同的任选，但她并不是喜欢每一个人。经过一番总结，她发现自己交往过的人可以进行如下分类：

- ✓ 不喜欢的人
- ✓ 魅力一般的人
- ✓ 极具魅力的人

海伦收集约会数据已经有了一段时间，她把这些数据存放在文本文件 `datingTestSet.txt` 中，每个样本数据占据一行，总共有 1000 行。

- ✓ 海伦收集的样本数据主要包含以下 3 种特征：
- ✓ 每年获得的飞行常客里程数
- ✓ 玩视频游戏所消耗时间百分比
- ✓ 每周消费的冰淇淋公升数

打开 txt 文本文件，数据格式如图所示。

1	40920	8.326976	0.953952	largeDoses	
2	14488	7.153469	1.673904	smallDoses	
3	26052	1.441871	0.805124	didntLike	不喜欢
4	75136	13.147394	0.428964	didntLike	
5	38344	1.669788	0.134296	didntLike	
6	72993	10.141740	1.032955	didntLike	
7	35948	6.830792	1.213192	largeDoses	极具魅力
8	42666	13.276369	0.543880	largeDoses	
9	67497	8.631577	0.749278	didntLike	
10	35483	12.273169	1.508053	largeDoses	
11	50242	3.723498	0.831917	didntLike	
12	63275	8.385879	1.669485	didntLike	
13	5569	4.875435	0.728658	smallDoses	魅力一般
14	51052	4.680098	0.625224	didntLike	
15	77372	15.299570	0.331351	didntLike	
16	43673	1.889461	0.191283	didntLike	
17	61364	7.516754	1.269164	didntLike	
18	69673	14.239195	0.261333	didntLike	
19	15669	0.000000	1.250185	smallDoses	
20	28488	10.528555	1.304844	largeDoses	
21	6487	3.540265	0.822483	smallDoses	
22	37708	2.991551	0.833920	didntLike	
23	22620	5.297865	0.638306	smallDoses	
24	28782	6.593803	0.187108	largeDoses	
25	19739	2.816760	1.686209	smallDoses	
26	36788	12.458258	0.649617	largeDoses	
27	5741	0.000000	1.656418	smallDoses	

(1) 准备数据：数据解析

在将上述特征数据输入到分类器前，必须将待处理的数据的格式改变为分类器可以接收的格式。分类器接收的数据是什么格式的？从上小结已经知道，要将数据分类两部分，即特征矩阵和对应的分类标记向量。在 `kNN_test02.py` 文件中创建名为 `file2matrix` 的函数，以此来处理输入格式问题。将 `datingTestSet.txt` 放到与 `kNN_test02.py` 相同目录下，编写代码如下：

```
# -*- coding: UTF-8 -*-
```

```
import numpy as np
```

```
"""
```

函数说明:打开并解析文件,对数据进行分类:1 代表不喜欢,2 代表魅力一般,3 代表极具魅力

Parameters:

filename - 文件名

Returns:

returnMat - 特征矩阵

classLabelVector - 分类 Label 向量

```
"""
```

```
def file2matrix(filename):
```

```
    #打开文件
```

```
    fr = open(filename)
```

```
    #读取文件所有内容
```

```
    arrayOLines = fr.readlines()
```

```
    #得到文件行数
```

```
    numberOfLines = len(arrayOLines)
```

```
    #返回的 NumPy 矩阵,解析完成的数据:numberOfLines 行,3 列
```

```
    returnMat = np.zeros((numberOfLines,3))
```

```
    #返回的分类标记向量
```

```
    classLabelVector = []
```

```
    #行的索引值
```

```
    index = 0
```

```
    for line in arrayOLines:
```

```
        #s.strip(rm), 当 rm 空时,默认删除空白符(包括'\n','\r','\t','')
```

```
        line = line.strip()
```

#使用 s.split(str="",num=string.count(str))将字符串根据'\t'分隔符进行切片。

```
        listFromLine = line.split('\t')
```

#将数据前三列提取出来,存放到 returnMat 的 NumPy 矩阵中,也就是特征矩阵

```
returnMat[index,:] = listFromLine[0:3]
```

#根据文本中标记的喜欢的程度进行分类,1 代表不喜欢,2 代表魅力一般,3 代表极具魅力

```
if listFromLine[-1] == 'didntLike':
    classLabelVector.append(1)
elif listFromLine[-1] == 'smallDoses':
    classLabelVector.append(2)
elif listFromLine[-1] == 'largeDoses':
    classLabelVector.append(3)
index += 1
return returnMat, classLabelVector
```

"""

函数说明:main 函数

Parameters:

无

Returns:

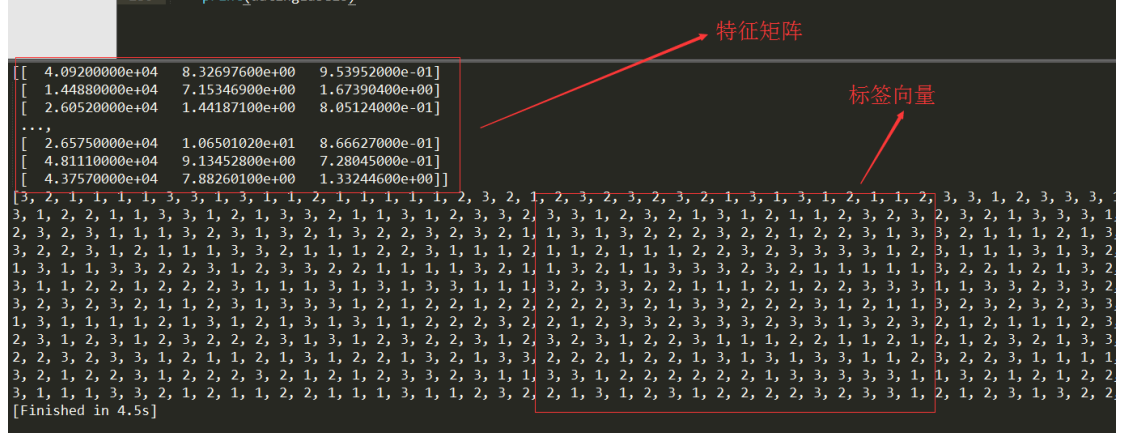
无

"""

```
if __name__ == '__main__':
    #打开的文件名
    filename = "datingTestSet.txt"
    #打开并处理数据
    datingDataMat, datingLabels = file2matrix(filename)
    print(datingDataMat)
    print(datingLabels)
```

运行上述代码，得到的数据解析结果如图所示。

```
279 """
280 if __name__ == '__main__':
281     #打开的文件名
282     filename = "datingTestSet.txt"
283     #打开并处理数据
284     datingDataMat, datingLabels = file2matrix(filename)
285     print(datingDataMat)
286     print(datingLabels)
```



可以看到，我们已经顺利导入数据，并对数据进行解析，格式化为分类器需要的数据格式。接着我们需要了解数据的真正含义。可以通过友好、直观的图形化的方式观察数据。

(2) 分析数据：数据可视化

在 kNN_test02.py 文件中编写名为 showdatas 的函数，用来将数据可视化。编写代码如下：

```
# -*- coding: UTF-8 -*-
```

```
from matplotlib.font_manager import FontProperties
import matplotlib.lines as mlines
import matplotlib.pyplot as plt
import numpy as np
```

```
"""
```

函数说明:打开并解析文件，对数据进行分类：1 代表不喜欢,2 代表魅力一般,3 代表极具魅力

Parameters:

filename - 文件名

Returns:

returnMat - 特征矩阵

classLabelVector - 分类 Label 向量

```

"""
def file2matrix(filename):
    #打开文件
    fr = open(filename)
    #读取文件所有内容
    arrayOLines = fr.readlines()
    #得到文件行数
    numberOfLines = len(arrayOLines)
    #返回的 NumPy 矩阵,解析完成的数据:numberOfLines 行,3 列
    returnMat = np.zeros((numberOfLines,3))
    #返回的分类标记向量
    classLabelVector = []
    #行的索引值
    index = 0
    for line in arrayOLines:
        #s.strip(rm), 当 rm 空时,默认删除空白符(包括'\n','\r','\t',' ')
        line = line.strip()
        #使用 s.split(str="",num=string.count(str))将字符串根据'\t'分隔符进行
切片。

        listFromLine = line.split('\t')
        #将数据前三列提取出来,存放到 returnMat 的 NumPy 矩阵中,也就
是特征矩阵

        returnMat[index,:] = listFromLine[0:3]
        #根据文本中标记的喜欢的程度进行分类,1 代表不喜欢,2 代表魅力
一般,3 代表极具魅力

        if listFromLine[-1] == 'didntLike':
            classLabelVector.append(1)
        elif listFromLine[-1] == 'smallDoses':
            classLabelVector.append(2)
        elif listFromLine[-1] == 'largeDoses':
            classLabelVector.append(3)
        index += 1

```



```
return returnMat, classLabelVector
```

```
"""
```

函数说明:可视化数据

Parameters:

datingDataMat - 特征矩阵

datingLabels - 分类 Label

Returns:

无

```
"""
```

```
def showdatas(datingDataMat, datingLabels):
```

```
    #设置汉字格式
```

```
    font = FontProperties(fname=r"c:\windows\fonts\simsun.ttc", size=14)
```

```
    #将 fig 画布分隔成 1 行 1 列,不共享 x 轴和 y 轴,fig 画布的大小为(13,8)
```

```
    #当 nrow=2,nclos=2 时,代表 fig 画布被分为四个区域,axs[0][0]表示第一  
    行第一个区域
```

```
    fig, axs = plt.subplots(nrows=2, ncols=2,sharex=False, sharey=False,  
figsize=(13,8))
```

```
    numberOfLabels = len(datingLabels)
```

```
    LabelsColors = []
```

```
    for i in datingLabels:
```

```
        if i == 1:
```

```
            LabelsColors.append('black')
```

```
        if i == 2:
```

```
            LabelsColors.append('orange')
```

```
        if i == 3:
```

```
            LabelsColors.append('red')
```

```
    #画出散点图,以 datingDataMat 矩阵的第一(飞行常客例程)、第二列(玩  
    游戏)数据画散点数据,散点大小为 15,透明度为 0.5
```

```
    axs[0][0].scatter(x=datingDataMat[:,0], y=datingDataMat[:,1],  
color=LabelsColors,s=15, alpha=.5)
```

```

#设置标题,x 轴 label,y 轴 label
axs0_title_text = axs[0][0].set_title(u'每年获得的飞行常客里程数与玩视
频游戏所消耗时间占比',FontProperties=font)
axs0_xlabel_text = axs[0][0].set_xlabel(u'每年获得的飞行常客里程数
',FontProperties=font)
axs0_ylabel_text = axs[0][0].set_ylabel(u'玩视频游戏所消耗时间占
',FontProperties=font)
plt.setp(axs0_title_text, size=9, weight='bold', color='red')
plt.setp(axs0_xlabel_text, size=7, weight='bold', color='black')
plt.setp(axs0_ylabel_text, size=7, weight='bold', color='black')

#画出散点图,以 datingDataMat 矩阵的第一(飞行常客里程)、第三列(冰
激凌)数据画散点数据,散点大小为 15,透明度为 0.5
axs[0][1].scatter(x=datingDataMat[:,0], y=datingDataMat[:,2],
color=LabelsColors,s=15, alpha=.5)
#设置标题,x 轴 label,y 轴 label
axs1_title_text = axs[0][1].set_title(u'每年获得的飞行常客里程数与每周
消费的冰激淋公升数',FontProperties=font)
axs1_xlabel_text = axs[0][1].set_xlabel(u'每年获得的飞行常客里程数
',FontProperties=font)
axs1_ylabel_text = axs[0][1].set_ylabel(u'每周消费的冰激淋公升数
',FontProperties=font)
plt.setp(axs1_title_text, size=9, weight='bold', color='red')
plt.setp(axs1_xlabel_text, size=7, weight='bold', color='black')
plt.setp(axs1_ylabel_text, size=7, weight='bold', color='black')

#画出散点图,以 datingDataMat 矩阵的第二(玩游戏)、第三列(冰激凌)数
据画散点数据,散点大小为 15,透明度为 0.5
axs[1][0].scatter(x=datingDataMat[:,1], y=datingDataMat[:,2],
color=LabelsColors,s=15, alpha=.5)
#设置标题,x 轴 label,y 轴 label
axs2_title_text = axs[1][0].set_title(u'玩视频游戏所消耗时间占比与每周
消费的冰激淋公升数',FontProperties=font)

```

```

    axs2_xlabel_text = axs[1][0].set_xlabel(u'玩视频游戏所消耗时间占比',FontProperties=font)
    axs2_ylabel_text = axs[1][0].set_ylabel(u'每周消费的冰激淋公升数',FontProperties=font)
    plt.setp(axs2_title_text, size=9, weight='bold', color='red')
    plt.setp(axs2_xlabel_text, size=7, weight='bold', color='black')
    plt.setp(axs2_ylabel_text, size=7, weight='bold', color='black')
    #设置图例
    didntLike = mlines.Line2D([], [], color='black', marker='.',
                                markersize=6, label='didntLike')
    smallDoses = mlines.Line2D([], [], color='orange', marker='.',
                                markersize=6, label='smallDoses')
    largeDoses = mlines.Line2D([], [], color='red', marker='.',
                                markersize=6, label='largeDoses')
    #添加图例
    axs[0][0].legend(handles=[didntLike,smallDoses,largeDoses])
    axs[0][1].legend(handles=[didntLike,smallDoses,largeDoses])
    axs[1][0].legend(handles=[didntLike,smallDoses,largeDoses])
    #显示图片
    plt.show()

```

"""

函数说明:main 函数

Parameters:

无

Returns:

无

"""

```
if __name__ == '__main__':
```

```
    #打开的文件名
```

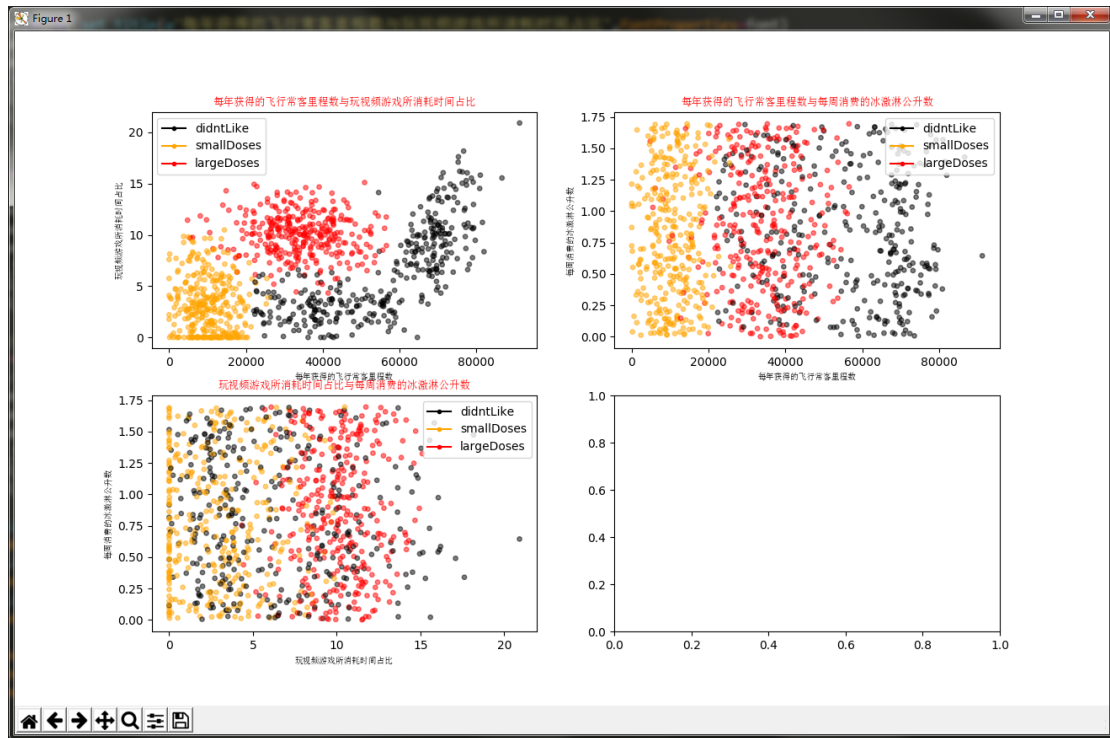
```
    filename = "datingTestSet.txt"
```

#打开并处理数据

```
datingDataMat, datingLabels = file2matrix(filename)
```

```
showdatas(datingDataMat, datingLabels)
```

运行上述代码，可以看到可视化结果如图所示。



通过数据可以很直观地发现数据的规律，比如以玩游戏所消耗时间占比与每年获得的飞行常客里程数，只考虑这二维的特征信息，给我的感觉就是海伦喜欢有生活质量的男人。为什么这么说呢？每年获得的飞行常客里程数表明，海伦喜欢能享受飞行常客奖励计划的男人，但是不能经常坐飞机，疲于奔波，满世界飞。同时，这个男人也要玩视频游戏，并且占一定时间比例。能到处飞，又能经常玩游戏的男人是什么样的男人？很显然，有生活质量，并且生活悠闲的人。我的分析，仅仅是通过可视化的数据总结的个人看法。我想，每个人的感受应该也是不尽相同。

(3) 准备数据：数据归一化

表 1 给出了四组样本，如果想要计算样本 3 和样本 4 之间的距离，可以使用欧拉公式计算。

表 1 约会网站样本数据

样本	玩游戏所耗时间百分比	每年获得的飞行常用里程数	每周消费的冰淇淋公升数	样本分类
1	0.8	400	0.5	1

2	12	134000	0.9	3
3	0	20000	1.1	2
4	67	32000	0.1	2

计算方法如图所示。

$$\sqrt{(0-67)^2 + (20000-32000)^2 + (1.1-0.1)^2}$$

我们很容易发现，上面方程中数字差值最大的属性对计算结果的影响最大，也就是说，每年获取的飞行常客里程数对于计算结果的影响将远远大于表 2.1 中其他两个特征-玩视频游戏所耗时间占比和每周消费冰淇淋公斤数的影响。而产生这种现象的唯一原因，仅仅是因为飞行常客里程数远大于其他特征值。但海伦认为这三种特征是同等重要的，因此作为三个等权重的特征之一，飞行常客里程数并不应该如此严重地影响到计算结果。

在处理这种不同取值范围的特征值时，我们通常采用的方法是将数值归一化，如将取值范围处理为 0 到 1 或者 -1 到 1 之间。下面的公式可以将任意取值范围的特征值转化为 0 到 1 区间内的值：

$$\text{newValue} = (\text{oldValue} - \text{min}) / (\text{max} - \text{min})$$

其中 min 和 max 分别是数据集中的最小特征值和最大特征值。虽然改变数值取值范围增加了分类器的复杂度，但为了得到准确结果，我们必须这样做。在 kNN_test02.py 文件中编写名为 autoNorm 的函数，用该函数自动将数据归一化。代码如下：

```
# -*- coding: UTF-8 -*-
import numpy as np

"""
```

函数说明:打开并解析文件，对数据进行分类：1 代表不喜欢,2 代表魅力一般,3 代表极具魅力

Parameters:

filename - 文件名

Returns:

returnMat - 特征矩阵

classLabelVector - 分类 Label 向量

```

"""
def file2matrix(filename):
    #打开文件
    fr = open(filename)
    #读取文件所有内容
    arrayOLines = fr.readlines()
    #得到文件行数
    numberOfLines = len(arrayOLines)
    #返回的 NumPy 矩阵,解析完成的数据:numberOfLines 行,3 列
    returnMat = np.zeros((numberOfLines,3))
    #返回的分类标记向量
    classLabelVector = []
    #行的索引值
    index = 0
    for line in arrayOLines:
        #s.strip(rm), 当 rm 空时,默认删除空白符(包括'\n','\r','\t',' ')
        line = line.strip()
        #使用 s.split(str="",num=string,cout(str))将字符串根据'\t'分隔符进行
切片。

        listFromLine = line.split('\t')
        #将数据前三列提取出来,存放到 returnMat 的 NumPy 矩阵中,也就是特征矩阵

        returnMat[index,:] = listFromLine[0:3]
        #根据文本中标记的喜欢的程度进行分类,1 代表不喜欢,2 代表魅力
一般,3 代表极具魅力

        if listFromLine[-1] == 'didntLike':
            classLabelVector.append(1)
        elif listFromLine[-1] == 'smallDoses':
            classLabelVector.append(2)
        elif listFromLine[-1] == 'largeDoses':
            classLabelVector.append(3)

        index += 1

    return returnMat, classLabelVector

```

"""

函数说明:对数据进行归一化

Parameters:

dataSet - 特征矩阵

Returns:

normDataSet - 归一化后的特征矩阵

ranges - 数据范围

minVals - 数据最小值

"""

def autoNorm(dataSet):

#获得数据的最小值

minVals = dataSet.min(0)

maxVals = dataSet.max(0)

#最大值和最小值的范围

ranges = maxVals - minVals

#shape(dataSet)返回 dataSet 的矩阵行列数

normDataSet = np.zeros(np.shape(dataSet))

#返回 dataSet 的行数

m = dataSet.shape[0]

#原始值减去最小值

normDataSet = dataSet - np.tile(minVals, (m, 1))

#除以最大和最小值的差,得到归一化数据

normDataSet = normDataSet / np.tile(ranges, (m, 1))

#返回归一化数据结果,数据范围,最小值

return normDataSet, ranges, minVals

"""

函数说明:main 函数

Parameters:

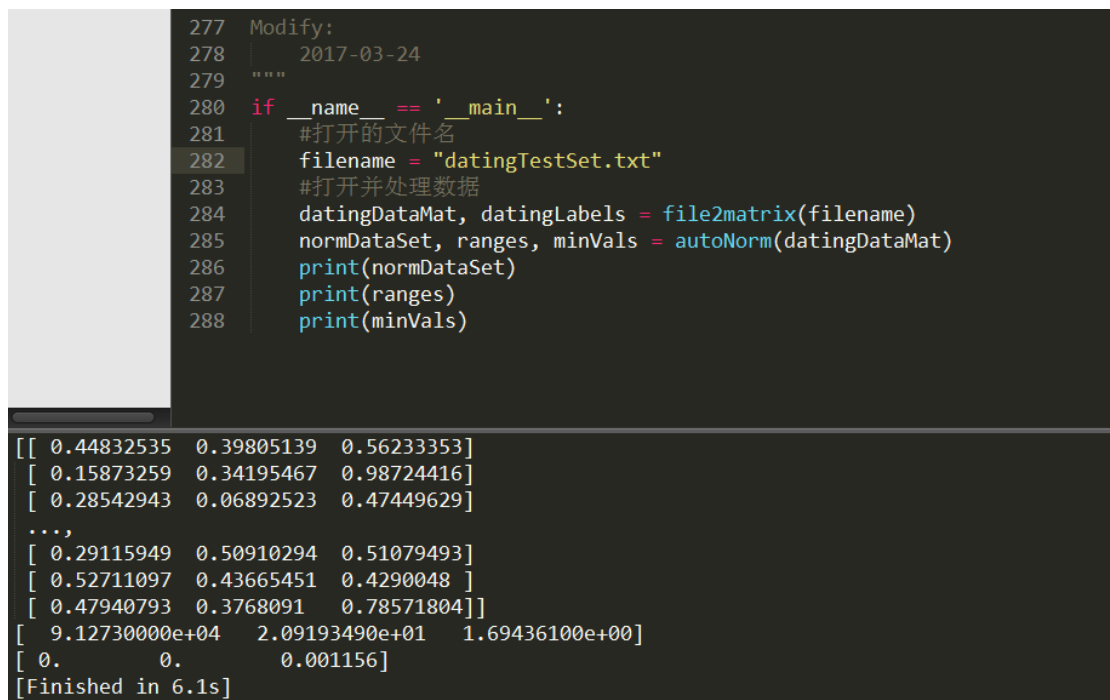
无

Returns:

无

```
"""
if __name__ == '__main__':
    #打开的文件名
    filename = "datingTestSet.txt"
    #打开并处理数据
    datingDataMat, datingLabels = file2matrix(filename)
    normDataSet, ranges, minVals = autoNorm(datingDataMat)
    print(normDataSet)
    print(ranges)
    print(minVals)
```

运行上述代码，得到结果如图所示。



```
277 Modify:
278 2017-03-24
279 """
280 if __name__ == '__main__':
281     #打开的文件名
282     filename = "datingTestSet.txt"
283     #打开并处理数据
284     datingDataMat, datingLabels = file2matrix(filename)
285     normDataSet, ranges, minVals = autoNorm(datingDataMat)
286     print(normDataSet)
287     print(ranges)
288     print(minVals)
```

```
[[ 0.44832535  0.39805139  0.56233353]
 [ 0.15873259  0.34195467  0.98724416]
 [ 0.28542943  0.06892523  0.47449629]
 ...,
 [ 0.29115949  0.50910294  0.51079493]
 [ 0.52711097  0.43665451  0.4290048 ]
 [ 0.47940793  0.3768091  0.78571804]]
[ 9.12730000e+04  2.09193490e+01  1.69436100e+00]
[ 0. 0. 0.001156]
[Finished in 6.1s]
```

从运行结果可以看到，我们已经顺利将数据归一化了，并且求出了数据的取值范围和数据的最小值，这两个值是在分类的时候需要用到的，直接先求解出来，也算是对数据预处理了。

(4) 测试算法：验证分类器

机器学习算法一个很重要的工作就是评估算法的正确率，通常我们只提供已

有数据的 90%作为训练样本来训练分类器，而使用其余的 10%数据去测试分类器，检测分类器的正确率。需要注意的是，10%的测试数据应该是随机选择的，由于海伦提供的数据并没有按照特定目的来排序，所以我么你可以随意选择 10%数据而不影响其随机性。

为了测试分类器效果，在 kNN_test02.py 文件中创建函数 datingClassTest，编写代码如下：

```
# -*- coding: UTF-8 -*-

import numpy as np
import operator

"""
函数说明:kNN 算法,分类器

Parameters:
    inX - 用于分类的数据(测试集)
    dataSet - 用于训练的数据(训练集)
    labes - 分类标记
    k - kNN 算法参数,选择距离最小的 k 个点

Returns:
    sortedClassCount[0][0] - 分类结果

"""

def classify0(inX, dataSet, labels, k):
    #numpy 函数 shape[0]返回 dataSet 的行数
    dataSetSize = dataSet.shape[0]
    #在列向量方向上重复 inX 共 1 次(横向),行向量方向上重复 inX 共
dataSetSize 次(纵向)
    diffMat = np.tile(inX, (dataSetSize, 1)) - dataSet
    #二维特征相减后平方
    sqDiffMat = diffMat**2
    #sum()所有元素相加,sum(0)列相加,sum(1)行相加
    sqDistances = sqDiffMat.sum(axis=1)
    #开方,计算出距离
```

```

distances = sqDistances**0.5
#返回 distances 中元素从小到大排序后的索引值
sortedDistIndices = distances.argsort()
#定一个记录类别次数的字典
classCount = {}
for i in range(k):
    #取出前 k 个元素的类别
    voteLabel = labels[sortedDistIndices[i]]
    #dict.get(key,default=None),字典的 get()方法,返回指定键的值,如果
    值不在字典中返回默认值。
    #计算类别次数
    classCount[voteLabel] = classCount.get(voteLabel,0) + 1
#python3 中用 items()替换 python2 中的 iteritems()
#key=operator.itemgetter(1)根据字典的值进行排序
#key=operator.itemgetter(0)根据字典的键进行排序
#reverse 降序排序字典
sortedClassCount =
sorted(classCount.items(),key=operator.itemgetter(1),reverse=True)
#返回次数最多的类别,即所要分类的类别
return sortedClassCount[0][0]

```

"""

函数说明:打开并解析文件,对数据进行分类:1 代表不喜欢,2 代表魅力一般,3 代表极具魅力

Parameters:

filename - 文件名

Returns:

returnMat - 特征矩阵

classLabelVector - 分类 Label 向量

"""

def file2matrix(filename):

```

#打开文件
fr = open(filename)
#读取文件所有内容
arrayOLines = fr.readlines()
#得到文件行数
numberOfLines = len(arrayOLines)
#返回的 NumPy 矩阵,解析完成的数据:numberOfLines 行,3 列
returnMat = np.zeros((numberOfLines,3))
#返回的分类标记向量
classLabelVector = []
#行的索引值
index = 0
for line in arrayOLines:
    #s.strip(rm), 当 rm 空时,默认删除空白符(包括'\n','\r','\t',' ')
    line = line.strip()
    #使用 s.split(str="",num=string,cout(str))将字符串根据'\t'分隔符进行
切片。

    listFromLine = line.split('\t')
    #将数据前三列提取出来,存放到 returnMat 的 NumPy 矩阵中,也就是特征矩阵

    returnMat[index,:] = listFromLine[0:3]
    #根据文本中标记的喜欢的程度进行分类,1 代表不喜欢,2 代表魅力
一般,3 代表极具魅力

    if listFromLine[-1] == 'didntLike':
        classLabelVector.append(1)
    elif listFromLine[-1] == 'smallDoses':
        classLabelVector.append(2)
    elif listFromLine[-1] == 'largeDoses':
        classLabelVector.append(3)
    index += 1
return returnMat, classLabelVector

```

"""

函数说明:对数据进行归一化

Parameters:

dataSet - 特征矩阵

Returns:

normDataSet - 归一化后的特征矩阵

ranges - 数据范围

minVals - 数据最小值

"""

def autoNorm(dataSet):

 #获得数据的最小值

 minVals = dataSet.min(0)

 maxVals = dataSet.max(0)

 #最大值和最小值的范围

 ranges = maxVals - minVals

 #shape(dataSet)返回 dataSet 的矩阵行列数

 normDataSet = np.zeros(np.shape(dataSet))

 #返回 dataSet 的行数

 m = dataSet.shape[0]

 #原始值减去最小值

 normDataSet = dataSet - np.tile(minVals, (m, 1))

 #除以最大和最小值的差,得到归一化数据

 normDataSet = normDataSet / np.tile(ranges, (m, 1))

 #返回归一化数据结果,数据范围,最小值

 return normDataSet, ranges, minVals

"""

函数说明:分类器测试函数

Parameters:

无

Returns:

normDataSet - 归一化后的特征矩阵

ranges - 数据范围

minVals - 数据最小值

"""

def datingClassTest():

 #打开的文件名

 filename = "datingTestSet.txt"

 #将返回的特征矩阵和分类向量分别存储到 datingDataMat 和
datingLabels 中

 datingDataMat, datingLabels = file2matrix(filename)

 #取所有数据的百分之十

 hoRatio = 0.10

 #数据归一化,返回归一化后的矩阵,数据范围,数据最小值

 normMat, ranges, minVals = autoNorm(datingDataMat)

 #获得 normMat 的行数

 m = normMat.shape[0]

 #百分之十的测试数据的个数

 numTestVecs = int(m * hoRatio)

 #分类错误计数

 errorCount = 0.0

 for i in range(numTestVecs):

 #前 numTestVecs 个数据作为测试集,后 m-numTestVecs 个数据作为
训练集

 classifierResult = classify0(normMat[i,:], normMat[numTestVecs:m,:],
 datingLabels[numTestVecs:m], 4)

 print(" 分 类 结 果 :%d\t 真 实 类 别 :%d" % (classifierResult,
datingLabels[i]))

 if classifierResult != datingLabels[i]:

 errorCount += 1.0

 print("错误率:%f%%" %(errorCount/float(numTestVecs)*100))

```
"""
```

函数说明:main 函数

Parameters:

无

Returns:

无

```
"""
```

```
if __name__ == '__main__':
```

```
    datingClassTest()
```

运行上述代码，得到结果如图所示。

```
271
272 Parameters:
273     无
274 Returns:
275     无
276
277 Modify:
278     2017-03-24
279 """
280 if __name__ == '__main__':
281     datingClassTest()
```

分类结果:1	真实类别:1
分类结果:3	真实类别:3
分类结果:1	真实类别:1
分类结果:2	真实类别:2
分类结果:2	真实类别:2
分类结果:1	真实类别:1
分类结果:1	真实类别:1
分类结果:3	真实类别:3
分类结果:2	真实类别:3
分类结果:1	真实类别:1
分类结果:2	真实类别:2
分类结果:1	真实类别:1
分类结果:3	真实类别:3
分类结果:3	真实类别:3
分类结果:2	真实类别:2
分类结果:1	真实类别:1
分类结果:1	真实类别:1

错误率:3.000000%

[Finished in 0.9s]

<http://blog.csdn.net/c406495762>

从结果中可以看出，错误率是 3%，这是一个相当不错的结果。我们可以改变函数 `datingClassTest` 内变量 `hoRatio` 和分类器 `k` 的值，检测错误率是否随着变量值的变化而增加。依赖于分类算法、数据集和程序设置，分类器的输出结果可

能有很大的不同。

(5) 使用算法：构建完整可用系统

我们可以给海伦一个小段程序，通过该程序海伦会在约会网站上找到某个人并输入他的信息。程序会给出她对男方喜欢程度的预测值。

```
# -*- coding: UTF-8 -*-

import numpy as np
import operator

"""
函数说明:kNN 算法,分类器

Parameters:
    inX - 用于分类的数据(测试集)
    dataSet - 用于训练的数据(训练集)
    labes - 分类标签
    k - kNN 算法参数,选择距离最小的 k 个点

Returns:
    sortedClassCount[0][0] - 分类结果
"""

def classify0(inX, dataSet, labels, k):
    #numpy 函数 shape[0]返回 dataSet 的行数
    dataSetSize = dataSet.shape[0]
    #在列向量方向上重复 inX 共 1 次(横向),行向量方向上重复 inX 共 dataSetSize
    次(纵向)
    diffMat = np.tile(inX, (dataSetSize, 1)) - dataSet
    #二维特征相减后平方
    sqDiffMat = diffMat**2
    #sum()所有元素相加,sum(0)列相加,sum(1)行相加
    sqDistances = sqDiffMat.sum(axis=1)
    #开方,计算出距离
    distances = sqDistances**0.5
    #返回 distances 中元素从小到大排序后的索引值
```

```

sortedDistIndices = distances.argsort()
#定一个记录类别次数的字典
classCount = {}
for i in range(k):
    #取出前 k 个元素的类别
    voteLabel = labels[sortedDistIndices[i]]
    #dict.get(key,default=None),字典的 get()方法,返回指定键的值,如果值不在字典中返回默认值。
    #计算类别次数
    classCount[voteLabel] = classCount.get(voteLabel,0) + 1
#python3 中用 items()替换 python2 中的 iteritems()
#key=operator.itemgetter(1)根据字典的值进行排序
#key=operator.itemgetter(0)根据字典的键进行排序
#reverse 降序排序字典
sortedClassCount = sorted(classCount.items(),key=operator.itemgetter(1),reverse=True)
#返回次数最多的类别,即所要分类的类别
return sortedClassCount[0][0]

```

"""

函数说明:打开并解析文件,对数据进行分类: 1 代表不喜欢,2 代表魅力一般,3 代表极具魅力

Parameters:

filename - 文件名

Returns:

returnMat - 特征矩阵

classLabelVector - 分类 Label 向量

"""

def file2matrix(filename):

 #打开文件


```

fr = open(filename)
#读取文件所有内容
arrayOLines = fr.readlines()
#得到文件行数
numberOfLines = len(arrayOLines)
#返回的 NumPy 矩阵,解析完成的数据:numberOfLines 行,3 列
returnMat = np.zeros((numberOfLines,3))
#返回的分类标签向量
classLabelVector = []
#行的索引值
index = 0
for line in arrayOLines:
    #s.strip(rm), 当 rm 空时,默认删除空白符(包括'\n','\r','\t',' ')
    line = line.strip()
    #使用 s.split(str="",num=string.count(str))将字符串根据'\t'分隔符进行切片。
    listFromLine = line.split('\t')
    #将数据前三列提取出来,存放到 returnMat 的 NumPy 矩阵中,也就是特征矩阵
    returnMat[index,:] = listFromLine[0:3]
    #根据文本中标记的喜欢的程度进行分类,1 代表不喜欢,2 代表魅力一般,3 代表极具魅力
    if listFromLine[-1] == 'didntLike':
        classLabelVector.append(1)
    elif listFromLine[-1] == 'smallDoses':
        classLabelVector.append(2)
    elif listFromLine[-1] == 'largeDoses':
        classLabelVector.append(3)
    index += 1
return returnMat, classLabelVector

```

"""

函数说明:对数据进行归一化

Parameters:

dataSet - 特征矩阵

Returns:

normDataSet - 归一化后的特征矩阵

ranges - 数据范围

minVals - 数据最小值

"""

def autoNorm(dataSet):

 #获得数据的最小值

 minVals = dataSet.min(0)

 maxVals = dataSet.max(0)

 #最大值和最小值的范围

 ranges = maxVals - minVals

 #shape(dataSet)返回 dataSet 的矩阵行列数

 normDataSet = np.zeros(np.shape(dataSet))

 #返回 dataSet 的行数

 m = dataSet.shape[0]

 #原始值减去最小值

 normDataSet = dataSet - np.tile(minVals, (m, 1))

 #除以最大和最小值的差,得到归一化数据

 normDataSet = normDataSet / np.tile(ranges, (m, 1))

 #返回归一化数据结果,数据范围,最小值

 return normDataSet, ranges, minVals

"""

函数说明:通过输入一个人的三维特征,进行分类输出

Parameters:

无

Returns:

无

```

"""
def classifyPerson():
    #输出结果
    resultList = ['讨厌','有些喜欢','非常喜欢']
    #三维特征用户输入
    precentTats = float(input("玩视频游戏所耗时间百分比:"))
    ffMiles = float(input("每年获得的飞行常客里程数:"))
    iceCream = float(input("每周消费的冰激淋公升数:"))
    #打开的文件名
    filename = "datingTestSet.txt"
    #打开并处理数据
    datingDataMat, datingLabels = file2matrix(filename)
    #训练集归一化
    normMat, ranges, minVals = autoNorm(datingDataMat)
    #生成 NumPy 数组,测试集
    inArr = np.array([precentTats, ffMiles, iceCream])
    #测试集归一化
    norminArr = (inArr - minVals) / ranges
    #返回分类结果
    classifierResult = classify0(norminArr, normMat, datingLabels, 3)
    #打印结果
    print("你可能%s 这个人" % (resultList[classifierResult-1]))

```

"""

函数说明:main 函数

Parameters:

无

Returns:

无

"""

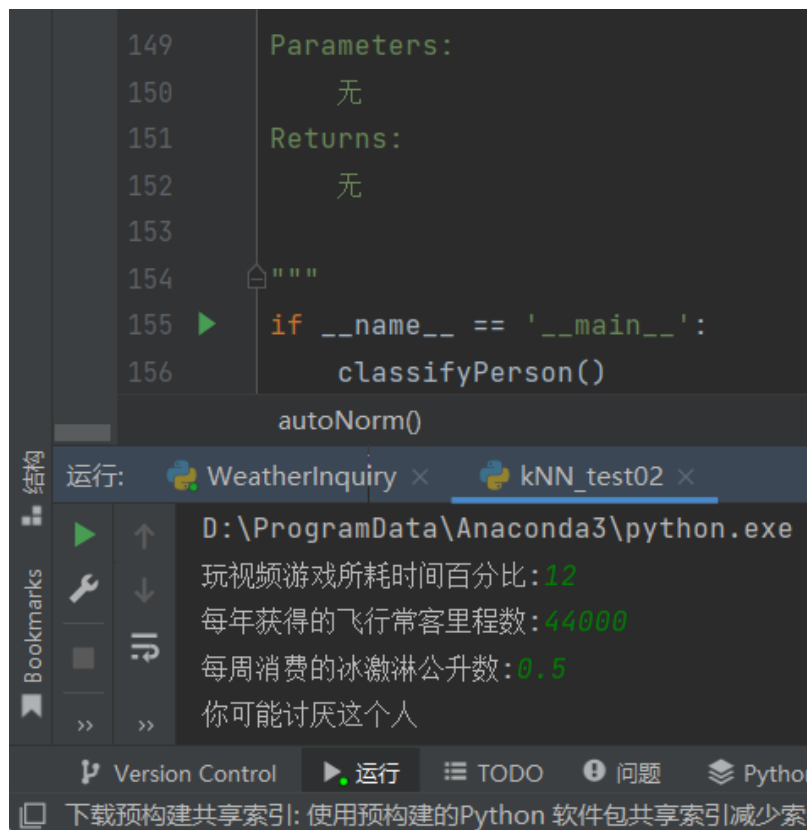
```

if __name__ == '__main__':

```

classifyPerson()

运行程序，并输入数据(12,44000,0.5)，预测结果是"你可能有些喜欢这个人"，也就是这个人魅力一般。一共有三个档次：讨厌、有些喜欢、非常喜欢，对应着不喜欢的人、魅力一般的人、极具魅力的人。结果如图所示。



```
149 Parameters:
150     无
151 Returns:
152     无
153
154 """
155 if __name__ == '__main__':
156     classifyPerson()

autoNorm()
```

运行: WeatherInquiry x kNN_test02 x

D:\ProgramData\Anaconda3\python.exe

玩视频游戏所耗时间百分比:12

每年获得的飞行常客里程数:44000

每周消费的冰激淋公升数:0.5

你可能讨厌这个人

四：实验要求

- 1、根据实验讲义完成相应实验内容，对比实验结果，调整参数，保证结果的正确性；
- 2、记录实验过程，撰写电子版实验报告，注意每张结果图片上需注明同学们的“学号+姓名”；
- 3、完成电子版实验报告后，调整报告格式内容，不限于字体、字号、标题、段落、图片大小等等，保证报告结果正确且美观，检查无误在学习通平台提交电子版实验报告文件，提交前修改电子版文件名称为“学号+姓名+实验1”；
- 4、打印纸质版实验报告进行保存，所有实验结束后整体装订成册，交给各班学习委员。